

---

# Beyond Static Allocation: Dynamic Sensitivity-Aware Fine-Tuning for Vision Transformers

---

Yuanyang Cao<sup>1</sup> Xichun Liu<sup>1</sup> Fuwei Zhang<sup>2</sup> Shangqi Deng<sup>1</sup> Ziyang Ren<sup>1</sup> Jianji Wang<sup>1</sup>

## Abstract

Existing Parameter-Efficient Fine-Tuning (PEFT) methods are fundamentally constrained by a static allocation paradigm, which overlooks the model’s evolving optimization priorities during training. To address this, we introduce Dynamic Adaptive Fine-tuning (DAF), a novel framework that periodically evaluates and reconfigures the trainable structure based on a context-aware decoupled sensitivity analysis. DAF employs a Rebuild-and-Refocus strategy to preserve learned knowledge by freezing outdated modules while decisively reallocating the parameter budget to newly identified critical regions. Extensive experiments on challenging vision benchmarks demonstrate that DAF significantly outperforms mainstream static PEFT methods and achieves superior performance and efficiency, particularly under extreme parameter budgets. Our work fundamentally challenges the static nature of the field, offering a more intelligent and efficient paradigm for adapting large pre-trained models. The code is available at <https://github.com/E-green11/DAF>.

## 1. Introduction

Large-scale pretrained vision models, particularly ViT (Dosovitskiy et al., 2020), have demonstrated remarkable generalization capabilities in many downstream visual tasks. The standard paradigm for adapting these powerful models to specific tasks is full fine-tuning. However, this approach requires storing a complete copy of the model for each task, and with the dramatic growth in the scale of the model (Zhai et al., 2022), the associated high storage and computational costs have become prohibitive. PEFT has emerged (Hu et al.,

2022; Jia et al., 2022) to address this challenge, which tunes only a small fraction of the model’s parameters, achieving performance even superior to that of full fine-tuning.

Existing PEFT methods largely follow a static allocation paradigm. One class of methods, such as Adapter (Houlsby et al., 2019), LoRA (Hu et al., 2022), and Visual Prompt Tuning (VPT) (Jia et al., 2022), typically relies on human prior knowledge to insert trainable modules at task-agnostic, fixed locations. Another class of methods attempts to adaptively select fine-tuning parameters for specific tasks (He et al., 2023). However, whether based on heuristic rules or a one-shot sensitivity analysis, these methods share a fundamental limitation: the locations and structures of all trainable parameters are determined once before training and remain unchanged throughout the entire fine-tuning process. Recently, although methods like VQT (Tu et al., 2023) and SynQT (Zhang et al., 2024a) have made new progress in how to utilize intermediate representations, the tuning structures they introduce are also fixed after training begins. This static assumption overlooks a critical fact: as the model progressively learns and adapts to the downstream task, its internal knowledge bottlenecks and optimization priorities dynamically evolve. A module that is crucial in the early stages of training may no longer be key to performance improvement later on; conversely, new bottlenecks will emerge. Therefore, any static allocation strategy cannot optimally utilize the limited parameter budget to adapt to the dynamic changes in the model’s own learning state.

The fundamental limitation of the static assumption becomes particularly prominent when contrasted with the operational mechanism of nature’s most efficient learning system—the biological brain. Learning in the nervous system is not a fixed, predetermined process. Instead, it is a highly dynamic process of remodeling. When faced with new knowledge or tasks, the brain does not uniformly activate all neurons. Rather, it employs a mechanism of sparse activation to selectively engage the specific neural circuits most relevant to the current stimulus (Poo et al., 2016). More importantly, through synaptic plasticity, the connection strengths between neurons are dynamically strengthened or weakened based on experience. This continuous structural adaptation is the key to achieving efficient, life-long learning in biological intelligence.

---

<sup>1</sup>State Key Laboratory of Human-Machine Hybrid Augmented Intelligence, Institute of Artificial Intelligence and Robotics, Xi’an Jiaotong University, Xi’an, Shaanxi, China <sup>2</sup>Institute of Artificial Intelligence, Beihang University, Beijing, China. Correspondence to: Jianji Wang <wangjianji@mail.xjtu.edu.cn>.

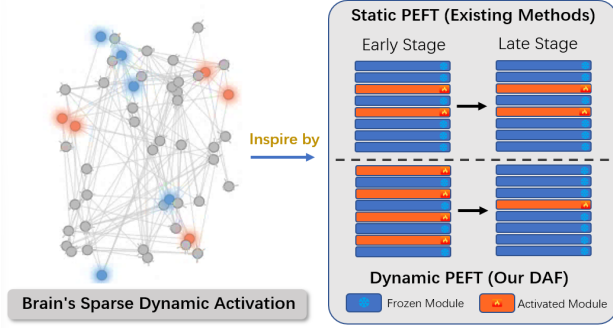


Figure 1. Inspired by the brain’s sparse and dynamic activation (Hill, 2025), the DAF framework periodically reconfigures its trainable structure. This contrasts with static PEFT methods that use fixed modules.

Inspired by this biological mechanism (Payeur et al., 2021), we argue that a more ideal fine-tuning framework should be able to dynamically perceive and adapt to the model’s evolution, as shown in Figure 1. We are the first to propose a novel dynamic reconfiguration paradigm for PEFT and design a framework named DAF. The core of DAF is a periodic perceive-decide-execute cycle. In each dynamic cycle, DAF initiates a three-stage process. First, it perceives the model’s state by employing our proposed context-aware decoupled sensitivity analysis method, which accurately evaluates the potential of the underlying backbone within the context of all previously learned knowledge. Subsequently, it decides on the most critical components for the current learning stage using a focused Top-K elite selection mechanism. Finally, DAF executes a decisive reconfiguration by adopting our proposed Rebuild-and-Refocus strategy, thereby concentrating all training resources on newly identified critical regions. This mechanism enables the model to shed the burden of training less relevant modules and adapt to new learning bottlenecks as quickly as possible. We conduct extensive experiments on multiple challenging public benchmarks. As illustrated in Figure 2, the compelling experimental results demonstrate that the DAF framework not only significantly outperforms mainstream static PEFT methods but also achieves superior performance and efficiency, particularly under extreme parameter budgets, validating the superiority of the dynamic paradigm. Furthermore, DAF exhibits exceptional versatility, generalizing effectively across diverse architectures, including Hierarchical Transformers and CNNs, where it even surpasses full fine-tuning. The main contributions consist of the following three aspects:

- We propose the first Dynamic Reconfiguration paradigm for PEFT, which challenges the static parameter allocation limitations of existing methods.
- We introduce the DAF framework featuring a Rebuild-

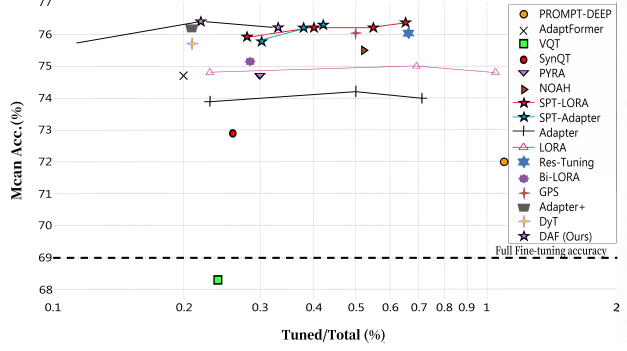


Figure 2. Performance comparison on the VTAB-1k benchmark. The DAF achieves superior performance against various static PEFT methods with remarkable parameter efficiency (tuning only 0.22% of parameters).

and-Refocus strategy. This mechanism preserves learned knowledge by freezing outdated modules while adaptively allocating the budget to new bottlenecks.

- We design a context-aware decoupled sensitivity analysis to mitigate signal noise, enabling precise assessment of the backbone’s potential.

**Conflict of Interest Disclosure.** The authors declare no financial conflicts of interest related to this work.

## 2. Related Work

**Parameter-Efficient Fine-Tuning.** Existing PEFT methods can be broadly categorized into three types based on how they introduce trainable parameters. Addition-based tuning adapts pretrained models by injecting new modules or prompts. Among these, Adapter (Houlsby et al., 2019) serially inserts small bottleneck layers, whereas AdaptFormer (Chen et al., 2022) places them in parallel with the FFN. This line of work has seen continued refinement, with Adapter+ (Steitz & Roth, 2024) optimizing the static adapter configuration and methods like Mona (Yin et al., 2025) and LoRand (Yin et al., 2023) proposing new adapter architectures for complex dense prediction tasks. Another popular branch is VPT (Jia et al., 2022), which adds learnable prompt tokens. This line of work has recently shifted from simply inserting prompts to modeling their cross-layer interaction and evolution. Recent cross-layer VPT variants further enhance prompt interactions by sharing task-relevant information across layers (Zhou et al., 2025), modeling prompt evolution with shared dynamical rules (Wang et al., 2026a), or introducing interpretable cross-layer prompt concepts (Wang et al., 2026b). Specification-based Tuning selectively fine-tunes a small subset of the model’s intrinsic parameters. For example, BitFit (Zaken et al., 2022) tunes only the bias terms, and SSF (Lian et al., 2022) learns to

scale and shift parameters. Reparameterization-based tuning methods, notably LoRA (Hu et al., 2022), approximate weight updates using trainable low-rank matrices, which can be merged at inference. This approach has also been recently enhanced by methods like DoRA (Liu et al., 2024a), which decomposes weights into magnitude and direction. Despite the success of these methods, their decisions on “where to fine-tune” largely rely on task-agnostic heuristics. To address this, some works explore adaptive parameter selection. For instance, GPS (Zhang et al., 2024c) and SPT (He et al., 2023) proposed identifying the most important parameters for a task via a one-time sensitivity analysis before fine-tuning begins. However, a common thread among all these methods (including heuristic-based, architecture-based, and selection-based) is their adherence to a static allocation paradigm. Our work fundamentally challenges this static assumption, arguing that the fine-tuning structure itself should evolve with the training process.

**Dynamic Model Adaptation.** Dynamics have been explored with different objectives. One direction targets inference efficiency, utilizing token pruning (DynamicViT (Rao et al., 2021), DVT (Wang et al., 2021)) or sparsification (Sparse-Tuning (Liu et al., 2024b), DyT (Zhao et al., 2024)). Another targets continual learning to mitigate forgetting across tasks (Wang et al., 2023; Wu et al., 2025). A third category optimizes training efficiency via lightweight side-networks (Head2Toe (Evci et al., 2022), LST (Sung et al., 2022), E<sup>3</sup>VA (Yin et al., 2024)) or learned queries (VQT (Tu et al., 2023), SynQT (Zhang et al., 2024a)). However, applying dynamics to the training process of a single task remains underexplored. A notable exception, called AdaLoRA (Zhang et al., 2023), adaptively prunes rank based on importance scores to reduce budget. In contrast, DAF introduces a reconfiguration paradigm that periodically freezes stabilized knowledge and reallocates the entire parameter budget to newly emerged bottlenecks. This approach treats fine-tuning as a dynamic trajectory rather than a fixed state, aiming for deeper model adaptation.

### 3. Method

#### 3.1. Preliminaries

**Vision Transformer.** A standard ViT model consists of a patch embedding layer and  $L$  stacked Transformer Blocks. Each Transformer Block  $l \in \{1, \dots, L\}$  typically includes a Multi-Head Self-Attention (MSA) module and a Feed-Forward Network (FFN). During fine-tuning, the vast majority of the ViT’s parameters, denoted as  $\theta_{\text{vit}}$ , remain frozen. LoRA is an efficient structured fine-tuning technique. For a pretrained weight matrix  $W_0 \in \mathbb{R}^{d \times k}$ , LoRA approximates its update,  $\Delta W$ , by introducing two trainable low-rank matrices,  $B \in \mathbb{R}^{d \times r}$  and  $A \in \mathbb{R}^{r \times k}$  (where the rank  $r \ll \min(d, k)$ ). During the forward pass, the output  $y$  of

the layer is computed as follows:

$$y = W_0 x + \Delta W x = W_0 x + B A x, \quad (1)$$

where only  $A$  and  $B$  are trainable.

#### 3.2. The DAF Framework: A Dynamic Reconfiguration Paradigm

The overall framework of DAF follows a periodic cycle of perceive-decide-execute, which we term Dynamic Reconfiguration. Unlike static methods that determine all trainable parameters at once before training, DAF repeatedly executes this cycle throughout the training process to achieve continuous optimization of the model’s fine-tuning structure. We illustrate the overall framework of DAF in Figure 3. As shown in the figure, the main training flow involves a pre-trained ViT where individual layers can be dynamically frozen or have their internal parameters activated. This activation state is not fixed at every dynamic analysis point (e.g., every  $E_{\text{interval}}$  epoch) and the Dynamic Reconfiguration module is triggered. This module executes a three-stage process: it first perceives the model’s state through dynamic sensitivity analysis, then decides on the new set of elite parameters to train, and finally executes the structural update via the Rebuild-and-Refocus strategy, which ensures the limited training budget is allocated to the most critical parts.

Assuming the total number of training epochs is  $E_{\text{total}}$  and the dynamic analysis interval is  $E_{\text{interval}}$ , DAF performs the following core operations at each dynamic analysis point  $t \in \{E_{\text{interval}}, 2E_{\text{interval}}, \dots\}$ :

- **Perceive:** Accurately assess the potential of the underlying backbone network within the complete learning context of the current model (see Section 3.3).
- **Decide:** Based on the assessment, employ a budget-based elite selection mechanism to identify the most important set of parameters  $\mathcal{P}_t^*$  for the current stage (see Section 3.4).
- **Execute:** Adopt a Rebuild-and-Refocus strategy to reconstruct the model’s structure, ensuring it only contains the tuning modules corresponding to the set  $\mathcal{P}_t^*$ , and perform weight migration (see Section 3.4).

#### 3.3. Context-Aware Decoupled Sensitivity Analysis

Providing a precise navigation signal for dynamic reconfiguration is key to DAF’s success. We found that directly analyzing a model that includes active LoRA modules introduces signal noise, as the training state such as high gradients of the LoRA modules themselves can dominate the analysis. To address this, we propose a context-aware decoupled dynamic analysis method. At each dynamic analysis point  $t$ , we define the main model being trained as

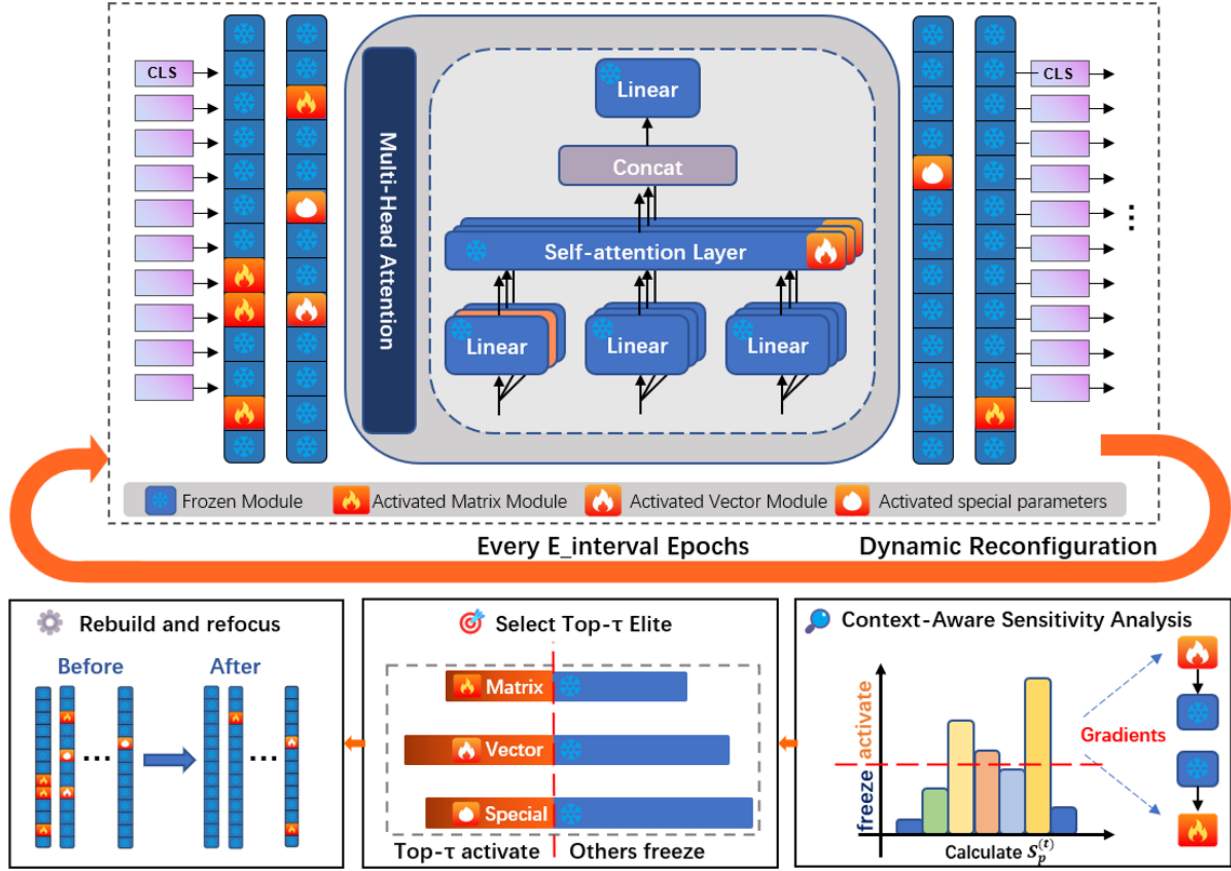


Figure 3. Overall framework of DAF. DAF periodically reconfigures the trainable structure of a pre-trained ViT during fine-tuning. At each dynamic analysis point, it executes a three-stage cycle: (1) Perceive the model state via Context-Aware Decoupled Sensitivity Analysis, (2) Decide on the most critical parameters (Matrix, Vector, and Special) using a budget-based elite selection, (3) Execute a Rebuild-and-Refocus strategy to update the set of active modules for the next training interval.

$\mathcal{M}_t$ , with parameters comprising the backbone weights  $\theta_{bb}^{(t)}$  and all existing LoRA module parameters  $\theta_{lora}^{(t)}$ . The analysis begins by temporarily freezing LoRA modules, we iterate through  $\mathcal{M}_t$  and set the parameters of all existing LoRA modules to a non-trainable state. Next, we perform an end-to-end gradient computation on the complete model. Since the LoRA modules still participate in the forward pass, they provide the correct, evolving learning context for the backbone’s gradient calculation. During the backward pass, the gradient flow is influenced by all LoRA modules but ultimately accumulates only on the backbone parameters, yielding a pure gradient information. For any backbone parameter  $w_p \in \theta_{bb}^{(t)}$ , its sensitivity  $s_p^{(t)}$  can be approximated by drawing inspiration from the model pruning method:

$$s_p^{(t)} = \left| \frac{\partial \mathcal{L}(\mathcal{D}; \mathcal{M}_t(\theta_{bb}^{(t)}, \theta_{lora}^{(t)}))}{\partial w_p} \cdot w_p \right|, \quad (2)$$

where  $\mathcal{L}$  is the task loss function and  $\mathcal{D}$  is a small batch of

data used for analysis. Because the gradients of LoRA parameters are disabled, this score reflects the purest potential signal of the backbone network. This dynamic, context-aware approach fundamentally differs from static methods that perform a one-shot analysis of the original unmodified backbone before any training begins.

### 3.4. Dynamic Reconfiguration: Budget-Based Elite Selection and Rebuild-and-Refocus Update

After obtaining the sensitivity scores for all backbone parameters, DAF employs a refined, budget-based elite selection mechanism instead of a simple global Top-K selection.

**Parameter Categorization.** To prevent the selection from being dominated by superior matrix parameters and ensure a functionally diverse set of tunable parameters is chosen, we first classify all backbone parameters to be analyzed into three categories: Matrix Parameters ( $\mathcal{P}_{mat}$ ), which are the primary candidates for LoRA tuning. Vector Parameters ( $\mathcal{P}_{vec}$ ), such as LayerNorm weights and biases, and Special

Parameters ( $\mathcal{P}_{\text{spec}}$ ), like `cls_token` and `pos.embed`.

**Budget-Based Elite Selection.** We first define an overall parameter budget, denoted by a ratio  $\tau$  (e.g.,  $\tau = 0.2$ ), which represents the target fraction of total possible PEFT parameters to be activated. To determine the specific parameter count budget for each category, this overall ratio  $\tau$  is combined with fixed allocation ratios for Matrix Parameters ( $\mathcal{P}_{\text{mat}}$ ), Vector Parameters ( $\mathcal{P}_{\text{vec}}$ ) and Special Parameters ( $\mathcal{P}_{\text{spec}}$ ). This partitioned budget strategy is crucial as it ensures a balanced selection of diverse parameter types, preventing the sensitivity scores of large-scale matrix parameters from dominating and overshadowing smaller, yet functionally critical, vector or special parameters. Within each category’s specifically allocated budget, we rank parameters by their sensitivity scores  $s_p^{(t)}$  and select the top-ranking ones to form the active sets  $\mathcal{A}_{\text{mat}}^{(t)}$ ,  $\mathcal{A}_{\text{vec}}^{(t)}$ , and  $\mathcal{A}_{\text{spec}}^{(t)}$ . The final set of parameters to be fine-tuned in the current cycle is the union  $\mathcal{P}_t^* = \mathcal{A}_{\text{mat}}^{(t)} \cup \mathcal{A}_{\text{vec}}^{(t)} \cup \mathcal{A}_{\text{spec}}^{(t)}$ . We add a constraint that only parameters with non-zero gradients are considered.

**First-Order View of Budgeted Dynamic Selection.** We further provide a first-order interpretation of the proposed elite selection rule. At reconfiguration stage  $t$ , let  $g_i^{(t)} = \partial \mathcal{L} / \partial w_i^{(t)}$  denote the context-aware decoupled gradient of a backbone parameter  $w_i^{(t)}$ , and let  $s_i^{(t)} = |g_i^{(t)} w_i^{(t)}|$  be its sensitivity score. DAF selects the active set under the category-wise budget family

$$\mathcal{F} = \{S : |S \cap \mathcal{P}_{\text{mat}}| \leq K_{\text{mat}}, |S \cap \mathcal{P}_{\text{vec}}| \leq K_{\text{vec}}, |S \cap \mathcal{P}_{\text{spec}}| \leq K_{\text{spec}}\}. \quad (3)$$

To connect this rule with local optimization, consider a sparse perturbation supported on any feasible set  $S \in \mathcal{F}$ :

$$\Delta_i = -\alpha \text{sign}(g_i^{(t)}) |w_i^{(t)}| \mathbf{1}[i \in S], \quad (4)$$

where  $\alpha > 0$  is a small step size. Under the first-order approximation

$$\mathcal{L}(\theta^{(t)} + \Delta) \approx \mathcal{L}(\theta^{(t)}) + \nabla \mathcal{L}(\theta^{(t)})^\top \Delta, \quad (5)$$

the predicted local loss decrease induced by  $S$  is

$$-\nabla \mathcal{L}(\theta^{(t)})^\top \Delta = \alpha \sum_{i \in S} |g_i^{(t)} w_i^{(t)}| = \alpha \sum_{i \in S} s_i^{(t)}. \quad (6)$$

Therefore, maximizing the predicted first-order descent under the same budget constraints reduces to

$$S_t^* = \arg \max_{S \in \mathcal{F}} \sum_{i \in S} s_i^{(t)}. \quad (7)$$

Since the objective is additive and the constraints are separable across parameter categories, this budgeted maximization

is solved by selecting the highest-sensitivity parameters within each category, which is exactly implemented by our category-wise Top- $K$  elite selection. Consequently, for any fixed feasible allocation  $S_{\text{static}} \in \mathcal{F}$ , the dynamically selected set satisfies

$$\sum_{i \in S_t^*} s_i^{(t)} \geq \sum_{i \in S_{\text{static}}} s_i^{(t)}, \quad (8)$$

where  $t$  denotes the current reconfiguration stage.

This result should be interpreted as a local first-order proxy rather than a global guarantee for the full non-convex objective. It nevertheless explains why dynamic allocation can be favored over static PEFT when the saliency pattern changes during training: DAF repeatedly matches the current dominant descent subspace, whereas a static method is constrained to a fixed trainable subspace throughout optimization.

**Rebuild-and-Refocus Update.** This mechanism ensures both learning continuity and adaptive resource allocation through a two-step process. First, for model reconstruction, we create a new, clean model instance,  $\mathcal{M}_{t+1}$ . This new model instantiates LoRA structures for all modules  $m$  in the union of the previous and current active sets,  $\mathcal{A}_{\text{mat}}^{(t-1)} \cup \mathcal{A}_{\text{mat}}^{(t)}$ . Second, we perform a meticulous weight migration to transfer knowledge from the old model  $\mathcal{M}_t$ . For any parameter  $w'_p$  in the new model  $\mathcal{M}_{t+1}$ :

$$w'_p \leftarrow \begin{cases} w_p, & \text{if } p \in \theta_{\text{bb}} \\ w_p, & \text{if } p \in m, m \in \mathcal{A}_{\text{mat}}^{(t-1)} \\ w_{\text{init}}, & \text{if } p \in m, m \in \mathcal{A}_{\text{mat}}^{(t)} \setminus \mathcal{A}_{\text{mat}}^{(t-1)} \end{cases} \quad (9)$$

where  $w_p$  is the corresponding parameter from the old model  $\mathcal{M}_t$ ,  $\theta_{\text{bb}}$  is the backbone parameter,  $w_{\text{init}}$  represents reinitialization and  $p \in m$  is in a LoRA module for  $m$ . Crucially, after weight migration, the Refocus step redefines the entire set of trainable parameters for the next training interval. For the matrix parameters, only the LoRA modules corresponding to the new active set  $\mathcal{A}_{\text{mat}}^{(t)}$  are enabled for training. Any LoRA module from the previous step that is no longer selected ( $m \in \mathcal{A}_{\text{mat}}^{(t-1)} \setminus \mathcal{A}_{\text{mat}}^{(t)}$ ) is immediately frozen, preserving its acquired knowledge while freeing up resources. In parallel, the update logic extends to the intrinsic backbone parameters: those selected for the new elite vector and special sets,  $\mathcal{A}_{\text{vec}}^{(t)}$  and  $\mathcal{A}_{\text{spec}}^{(t)}$ , are marked as trainable, while any previously trained vector or special parameters that are no longer part of the elite sets are frozen.

Specifically, we employ a partial warm restart strategy during this transition. While the model weights are inherited (warm start) to preserve learned knowledge, the optimizer states are explicitly reset. Ideally, the historical momentum vectors represent the optimization trajectory within the

previous parameter subspace. When the model topology is reconfigured, the optimization landscape shifts fundamentally. Applying outdated momentum to the new subspace can provide misleading gradient directions, hindering adaptation. Therefore, resetting the optimizer acts as a rectification mechanism, allowing the model to rapidly align with the gradient flow of the new topology. Empirically, as shown in Appendix C, although this reset causes momentary loss fluctuations, it effectively functions as structural annealing, helping the model escape local minima and converge to a significantly lower final loss compared to maintaining static momentum. This comprehensive process ensures that knowledge from the backbone and all previously learned modules is inherited, while training resources are decisively refocused on the newly identified critical regions across all parameter types. Through this cycle, DAF ensures that its fine-tuning structure is tailored to the current learning state at each stage, thereby achieving maximal adaptability. The complete process of the DAF is detailed in Algorithm 1.

**Zero-Overhead Inference.** Post-training, DAF merges all learned LoRA parameters into the backbone via  $W_{\text{final}} = W_0 + BA$ . The final model is architecturally identical to the original ViT, ensuring zero additional storage or computational cost during inference.

---

**Algorithm 1** The DAF Framework
 

---

**Require:** Pretrained model  $\mathcal{M}_0$ , Total epochs  $E$ , Interval  $\Delta T$ , Budget  $\tau$ .

- 1: **Initialize:** Compute sensitivity  $\mathcal{S}^{(0)}$  on  $\mathcal{M}_0$ .
  - 2: Split candidates into  $\mathcal{P}_{\text{mat}}$ ,  $\mathcal{P}_{\text{vec}}$ , and  $\mathcal{P}_{\text{spec}}$ .
  - 3: Determine initial active set  $\Omega_0 \leftarrow \text{Top}_\tau(\mathcal{S}^{(0)})$ .
  - 4: Construct  $\mathcal{M}$  with LoRA for  $\Omega_0$ ; Init Optimizer  $\mathcal{O}$ .
  - 5: **for** epoch  $t = 1$  **to**  $E$  **do**
  - 6:   Update  $\mathcal{M}$  via  $\mathcal{O}$ :  $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\mathcal{D}_{\text{train}})$ .
  - 7:   **if**  $t \pmod{\Delta T} = 0$  **then**
  - 8:     *// Phase 1: Perceive (Context-Aware Analysis)*
  - 9:     Freeze LoRA gradients while keeping LoRA modules in the forward path.
  - 10:      $g_w \leftarrow \nabla_w \mathcal{L}(\mathcal{M})|_{\theta_{\text{loa}} \text{ fixed}}$  for  $w \in \theta_{\text{bb}}$ .
  - 11:      $\mathcal{S}^{(t)} \leftarrow \{|g_w \cdot w| \mid w \in \theta_{\text{bb}}\}$ .  $\triangleright$  See Eq. (2)
  - 12:     *// Phase 2: Decide (Elite Selection)*
  - 13:      $\Omega_t \leftarrow \text{Top}_\tau(\mathcal{S}^{(t)})$ .  $\triangleright$  Select top- $\tau$  parameters (Mat, Vec, Spec)
  - 14:     *// Phase 3: Execute (Rebuild-and-Refocus)*
  - 15:      $\mathcal{M}_{\text{new}} \leftarrow \text{Reconstruct}(\Omega_{t-1} \cup \Omega_t)$ .
  - 16:      $\theta_{\text{new}} \leftarrow \text{MigrateWeights}(\theta_{\text{old}}, \Omega_{t-1}, \Omega_t)$ .  $\triangleright$  See Eq. (9)
  - 17:      $\mathcal{M} \leftarrow \text{SetActive}(\mathcal{M}_{\text{new}}, \Omega_t)$ .  $\triangleright$  Freeze  $\Omega_{t-1} \setminus \Omega_t$
  - 18:     Update  $\mathcal{O}$  with partial warm restart for  $\Omega_t$ .
  - 19:   **end if**
  - 20: **end for**
  - 21: **return**  $\mathcal{M}$
- 

## 4. Experiments

### 4.1. Experimental Setup

**Datasets.** To ensure a comprehensive evaluation, we conduct experiments across a diverse range of visual tasks. For classification, we utilize the standard Fine-Grained Visual Classification (FGVC) benchmark and the large-scale Visual Task Adaptation Benchmark (VTAB-1k) (Zhai et al., 2019). To further demonstrate the versatility of DAF on complex dense prediction tasks, we extend our evaluation to Object Detection on MS COCO (Lin et al., 2014) and Semantic Segmentation on ADE20K (Zhou et al., 2017). Further details are provided in Appendix A.

**Implementation Details.** Our primary experiments utilize a ViT-B/16 backbone pre-trained on ImageNet-21k. To verify architectural generalization, we also employ Hierarchical Transformers (Swin-B/L) and CNN-based architectures (ConvNeXt-B). We use the AdamW optimizer on NVIDIA RTX4090 GPUs. Detailed hyperparameters are listed in Appendix B.

**Baselines.** We compare DAF against a comprehensive suite of PEFT methods, ranging from classic approaches to the latest SOTA. These baselines cover three primary paradigms: (1) Addition-based methods, such as Adapter (Houlsby et al., 2019), AdaptFormer (Chen et al., 2022); (2) Reparameterization-based methods, such as LoRA (Hu et al., 2022); and (3) Prompt-based methods, such as VPT (Jia et al., 2022), NOAH (Zhang et al., 2024b). Crucially, we include strong competitors from 2023-2024, such as SPT (He et al., 2023), VQT (Tu et al., 2023), Res-Tuning (Jiang et al., 2023), LoRand (Yin et al., 2023), Bi-LoRA (Jie et al., 2023), PYRA (Xiong et al., 2024), DyT (Zhao et al., 2024), AdaLoRA (Zhang et al., 2023), Adapter+ (Steitz & Roth, 2024), GPS (Zhang et al., 2024c), SynQT (Zhang et al., 2024a), CaRA (Veeramacheneni et al., 2025), GLoRA (Chavan et al., 2023), MLAe (Wang et al., 2024) and Mona (Yin et al., 2025). A Static DAF baseline is introduced to isolate the benefits of our dynamic mechanism.

### 4.2. Main Results on Standard Benchmarks

We first compare DAF with baseline methods under the standard setting using a supervised pre-trained ViT-B/16 backbone. The average accuracy on FGVC and VTAB-1k is presented in Table 1.

As delineated in Table 1, the proposed DAF framework achieves superior performance, outperforming the static PEFT baselines. This highlights the significant advantage of dynamically reallocating parametric resources during training over a fixed, pre-determined fine-tuning strategy. Furthermore, the performance gap between DAF and our Static DAF baseline directly validates that the observed

Table 1. Overall performance comparison on FGVC and VTAB-1k benchmarks (ViT-B/16, ImageNet-21k pre-trained). Accuracy is Top-1 Avg. (%). ‘Tuned/Total’ denotes the fraction of trainable parameters. We highlight the **best** and the second-best results. Recent SOTA methods (2023-2025) are included as per reviewer feedback.

| Method                              | FGVC            |               | VTAB-1k         |             |             |             |               |
|-------------------------------------|-----------------|---------------|-----------------|-------------|-------------|-------------|---------------|
|                                     | Tuned/Total (%) | Mean Acc. (%) | Tuned/Total (%) | Natural     | Specialized | Structured  | Mean Acc. (%) |
| Full Fine-tuning                    | 100             | 88.5          | 100             | 75.9        | 83.4        | 47.6        | 69.0          |
| <i>Static PEFT Baselines</i>        |                 |               |                 |             |             |             |               |
| Adapter-8                           | 0.39            | 85.5          | 0.23            | 79.0        | 84.1        | 58.5        | 73.9          |
| Adapter-32                          | 0.95            | 85.6          | 0.71            | 79.6        | 84.0        | 58.3        | 74.0          |
| LoRA-8                              | 0.55            | 86.0          | 0.23            | 79.5        | 84.6        | 60.5        | 74.9          |
| LoRA-16                             | 0.90            | 84.8          | 0.69            | 79.8        | 84.9        | 60.2        | 75.0          |
| VPT-Deep                            | 0.35            | 83.8          | 0.32            | 78.5        | 82.4        | 55.0        | 72.0          |
| AdaptFormer                         | 0.23            | 86.1          | <b>0.20</b>     | 80.5        | 84.9        | 58.8        | 74.7          |
| NOAH                                | 0.50            | 89.2          | 0.52            | 80.2        | 84.9        | <u>61.3</u> | 75.5          |
| <i>Recent Static SOTA Baselines</i> |                 |               |                 |             |             |             |               |
| VQT                                 | 0.30            | 82.5          | 0.24            | 76.0        | 82.5        | 46.3        | 68.3          |
| SPT-Adapter                         | 0.41            | 89.5          | 0.30            | 81.3        | 85.3        | 60.8        | 75.8          |
| SPT-LoRA                            | 0.41            | 89.3          | 0.31            | 81.5        | <u>85.6</u> | 60.7        | 75.9          |
| Res-Tuning                          | 0.79            | 90.0          | 0.64            | 82.3        | 85.4        | 61.2        | 76.2          |
| Bi-LoRA                             | 0.24            | 89.3          | 0.28            | 81.1        | 84.4        | 60.5        | 75.3          |
| SynQT                               | 0.30            | 84.7          | 0.26            | 78.0        | 84.4        | 56.2        | 72.9          |
| AdaLoRA                             | 0.49            | 89.4          | 0.24            | 81.8        | 85.4        | 60.6        | 75.9          |
| PYRA                                | 0.34            | 86.2          | 0.30            | 79.1        | 84.4        | 60.6        | 74.7          |
| GPS                                 | 0.77            | 90.0          | 0.50            | <u>82.4</u> | 85.5        | 60.6        | 76.2          |
| Adapter+(r=1)                       | <u>0.22</u>     | <u>90.1</u>   | 0.23            | <b>83.2</b> | 85.5        | 60.1        | <u>76.3</u>   |
| DyT (r=0.5)                         | 0.23            | 90.0          | 0.23            | 80.8        | <u>85.6</u> | 60.7        | 75.7          |
| <i>Our Methods (Dynamic)</i>        |                 |               |                 |             |             |             |               |
| Static DAF (Ours)                   | <b>0.21</b>     | 89.5          | <u>0.22</u>     | 81.5        | 85.2        | 60.8        | 75.8          |
| <b>DAF (Ours)</b>                   | <b>0.21</b>     | <b>90.2</b>   | <u>0.22</u>     | 82.0        | <b>85.9</b> | <b>61.4</b> | <b>76.4</b>   |

gains are attributable to the dynamic reconfiguration mechanism. To further substantiate the stability and convergence superiority of this dynamic process, we provide a visual analysis of training loss trajectories in Appendix C. For a more granular analysis, we present the detailed per-task results on both benchmarks in Appendix D and provide the corresponding dynamic behavior visualizations for each task in Appendix E. In addition to model performance, we also evaluate the computational efficiency of our method. Crucially, we maintain a low memory footprint by avoiding the storage of backbone optimizer states during the sensitivity analysis. As detailed in Appendix F and Appendix G, DAF demonstrates a competitive efficiency profile, achieving its superior performance with a minimal parameter budget and no additional inference overhead.

### 4.3. Versatility in Self-Supervised Pre-training Paradigms

To rigorously test the versatility and robustness of DAF, we evaluated DAF on ViT-B/16 models pre-trained with two distinct self-supervised paradigms: Masked Autoencoders (MAE) and Momentum Contrast v3 (MoCo v3). Theoretically, the specialized features learned through these paradigms can result in highly varied parameter sensitivity

ties across different downstream tasks. This characteristic poses a significant challenge for static PEFT methods, as their fixed allocation of trainable parameters may struggle to adapt to such shifting optimization requirements.

This is precisely the scenario where DAF’s dynamic reconfiguration paradigm is designed to excel. By periodically re-evaluating and re-allocating trainable parameters, DAF can fluidly adapt to a model’s evolving optimization priorities. The results in Table 3 provide strong evidence for this approach. DAF consistently outperforms the static baselines on both self-supervised backbones, suggesting its dynamic mechanism is uniquely suited to unlocking their full potential by effectively navigating their complex optimization landscapes.

### 4.4. Performance on Complex Visual Tasks

**Object Detection on MS COCO.** We utilized the MS COCO 2017 dataset (Lin et al., 2014) and employed Cascade Mask R-CNN (Cai & Vasconcelos, 2018; He et al., 2017) as the object detection framework. Following standard practices for prediction tasks, we adopted a Swin-B backbone initialized from ImageNet-21k weights. As reported in Table 2 (Left), DAF achieves a remarkable 53.5  $AP^{box}$  and 46.1  $AP^{mask}$ . Notably, DAF surpasses the static

Table 2. Results on complex visual tasks. Left: Object Detection and Instance Segmentation on COCO val2017. Right: Semantic Segmentation on ADE20K. We include recent SOTA methods on Swin backbones.

| Object Detection (COCO) |               |             |             |             | Semantic Segmentation (ADE20K) |               |             |             |
|-------------------------|---------------|-------------|-------------|-------------|--------------------------------|---------------|-------------|-------------|
| Method                  | Backbone      | Params      | $AP^{box}$  | $AP^{mask}$ | Method                         | Backbone      | Params      | mIoU        |
| Full Fine-tuning        | Swin-B        | 89M         | 52.4        | 45.1        | Full Fine-tuning               | Swin-L        | 198M        | 51.1        |
| BitFit                  | Swin-B        | 0.2M        | 50.1        | 43.6        | BitFit                         | Swin-L        | 0.3M        | 48.3        |
| NormTuning              | Swin-B        | 0.1M        | 50.1        | 43.5        | NormTuning                     | Swin-L        | 0.1M        | 47.9        |
| Partial-1               | Swin-B        | 13.0M       | 50.6        | 43.7        | Partial-1                      | Swin-L        | 28.8M       | 47.4        |
| Adapter                 | Swin-B        | 3.2M        | 52.1        | 45.0        | Adapter                        | Swin-L        | 4.6M        | 50.8        |
| LoRA                    | Swin-B        | 3.1M        | 50.4        | 43.9        | LoRA                           | Swin-L        | 4.6M        | 50.3        |
| AdaptFormer             | Swin-B        | 1.6M        | 51.7        | 44.6        | AdaptFormer                    | Swin-L        | 2.3M        | 50.8        |
| LoRand                  | Swin-B        | 2.4M        | 51.1        | 44.1        | LoRand                         | Swin-L        | 3.6M        | 50.7        |
| LoRand++                | Swin-B        | 9.3M        | 51.5        | 44.4        | LoRand++                       | Swin-L        | 14.2M       | 51.9        |
| CaRA                    | Swin-B        | 1.1M        | 51.9        | 44.7        | CaRA                           | Swin-L        | 2.0M        | 51.0        |
| GLoRA                   | Swin-B        | 3.2M        | 52.3        | 44.9        | GLoRA                          | Swin-L        | 5.4M        | 51.3        |
| MLAE                    | Swin-B        | 3.1M        | 52.5        | 45.0        | MLAE                           | Swin-L        | 5.2M        | 51.4        |
| Mona                    | Swin-B        | 4.2M        | 53.4        | 46.0        | Mona                           | Swin-L        | 5.1M        | 51.4        |
| <b>DAF (Ours)</b>       | <b>Swin-B</b> | <b>2.1M</b> | <b>53.5</b> | <b>46.1</b> | <b>DAF (Ours)</b>              | <b>Swin-L</b> | <b>3.7M</b> | <b>52.0</b> |

Table 3. Performance comparison on the VTAB-1k benchmark across different ViT-B/16 self-supervised backbones.

|         | Method            | Natural     | Specialized | Structured  | Mean (%)    |
|---------|-------------------|-------------|-------------|-------------|-------------|
| MAE     | Full Fine-tuning  | 59.3        | 79.7        | 53.8        | 64.3        |
|         | LoRA-16           | 57.3        | 77.1        | 59.9        | 64.8        |
|         | Adapter-32        | 55.3        | 78.8        | 53.3        | 62.5        |
|         | SPT-Adapter       | 64.8        | 82.4        | 60.4        | 69.2        |
|         | SPT-LoRA          | 63.8        | 81.6        | 60.0        | 68.5        |
|         | BIAS              | 54.6        | 75.7        | 47.7        | 59.3        |
|         | VPT-Deep          | 50.8        | 76.4        | 37.3        | 54.8        |
|         | VQT               | 56.6        | 78.6        | 43.4        | 59.5        |
|         | SynQT             | 66.0        | 82.6        | 58.2        | 68.9        |
|         | <b>DAF (Ours)</b> | <b>65.7</b> | <b>82.7</b> | <b>61.4</b> | <b>69.9</b> |
| MoCo v3 | Full Fine-tuning  | 72.0        | 84.7        | 52.0        | 69.6        |
|         | LoRA-16           | 16.0        | 64.0        | 48.7        | 42.9        |
|         | Adapter-32        | 74.2        | 82.7        | 47.7        | 68.2        |
|         | SPT-Adapter       | 76.1        | 84.9        | 60.1        | 73.7        |
|         | SPT-LoRA          | 76.5        | 85.4        | 63.0        | 75.0        |
|         | BIAS              | 72.9        | 81.1        | 53.4        | 69.2        |
|         | <b>DAF (Ours)</b> | <b>76.7</b> | <b>85.9</b> | <b>63.7</b> | <b>75.4</b> |

LoRA baseline by a substantial margin of 3.1  $AP^{box}$  and 2.2  $AP^{mask}$ , while using only 2.1M trainable parameters. This shows that our dynamic budget allocation effectively captures the multi-scale object-centric features essential for precise localization.

**Semantic Segmentation on ADE20K.** We further extended our evaluation to the ADE20K dataset (Zhou et al., 2017) using the UperNet (Xiao et al., 2018) framework with a stronger Swin-L backbone. As shown in Table 2 (Right), DAF attains a best mIoU of 52.0%. This performance significantly outperforms the LoRA baseline (50.3%) by 1.7% and surpasses the previous best method LoRand++ with fewer parameters.

These results align with observations in Mona (Yin et al.,

2025) and LoRand (Yin et al., 2023) that standard static PEFT methods often struggle with the complex spatial dependencies required in these tasks. By introducing the dynamic reconfiguration paradigm, DAF effectively bridges this gap. Additionally, we provide a comprehensive analysis of DAF’s generalization capabilities on diverse backbones (e.g., ConvNeXt) for classification tasks in Appendix I, enabling flexible adaptation to diverse visual structures while maintaining superior parameter efficiency compared to both full fine-tuning and existing PEFT counterparts.

#### 4.5. Ablation Studies

We evaluate the contribution of DAF’s core components: Dynamic Reconfiguration (DR), Context-Aware Analysis (CAA), and Rebuild-and-Refocus (RR), with results summarized in Table 4. (1) Effectiveness of DR: DAF outperforms the “Static DAF” baseline (76.4% vs. 75.8%), validating that adapting to the model’s evolving priorities is superior to fixed allocation. The robustness of this gain against various static hyperparameter configurations is further verified in Appendix J. (2) Necessity of CAA: The performance drop in “DAF-Naive” (74.6%) confirms that evaluating sensitivity on the current adapted model is crucial, whereas only analyzing a pristine backbone yields inaccurate navigation signals. (3) Benefit of RR: “DAF-Accumulate” (75.3%) lags behind our method, demonstrating that merely accumulating parameters restricts plasticity. The RR strategy decisively reallocates the budget to emerging bottlenecks, ensuring maximum optimization agility compared to simple accumulation.

**Impact of Dynamic Update Frequency and Budget.** We study the effect of the reconfiguration interval  $E_{interval}$  and the parameter budget ratio  $\tau$ . As shown in Table 5, updates that are too frequent (e.g., every 5 epochs) can lead to train-

Table 4. Ablation study on core components. **DR**: Dynamic Reconfiguration, **CAA**: Context-Aware Analysis, **RR**: Rebuild-and-Refocus.

| Method            | DR | CAA | RR | Mean (%)    |
|-------------------|----|-----|----|-------------|
| Static DAF        |    | ✓   | ✓  | 75.8        |
| DAF-Naive         | ✓  |     | ✓  | 74.6        |
| DAF-Accumulate    | ✓  | ✓   |    | 75.3        |
| <b>DAF (Ours)</b> | ✓  | ✓   | ✓  | <b>76.4</b> |

ing instability, while updates that are too sparse (e.g., every 50 epochs) fail to capture the evolving training dynamics effectively. An interval of 10 epochs appears to offer the best trade-off. For the parameter budget  $\tau$ , shown in Table 6, we observe that performance peaks at  $\tau = 0.2$  and then slightly degrades as the budget continues to increase. This phenomenon can be explained by the nature of the sensitivity analysis. A relatively small subset of backbone parameters exhibits high sensitivity (i.e., large gradients), while the vast majority have very low sensitivity scores. A budget larger than optimal (e.g.,  $\tau = 0.3$  or  $0.4$ ) forces the elite selection mechanism to include parameters with minimal sensitivity. Allocating training resources to these non-critical parameters is counterproductive; it can introduce noise into the optimization process and diverts the training budget away from the components that are most crucial for adaptation, leading to a marginal decline in performance. Therefore,  $\tau = 0.2$  strikes an excellent balance, capturing a sufficient set of critical parameters for effective adaptation without wasting resources on less relevant ones.

Table 5. Impact of dynamic update frequency  $E_{\text{interval}}$  on VTAB-1k dataset.

| $E_{\text{interval}}$ (epochs) | 5    | 10          | 25   | 50   |
|--------------------------------|------|-------------|------|------|
| Mean Acc. (%)                  | 76.0 | <b>76.4</b> | 76.1 | 75.9 |

Table 6. Impact of parameter budget  $\tau$  on VTAB-1k.

| $\tau$        | 0.1  | 0.2         | 0.3  | 0.4  |
|---------------|------|-------------|------|------|
| Mean Acc. (%) | 75.7 | <b>76.4</b> | 76.2 | 76.1 |

**Robustness to Sensitivity Batch Num.** We further investigated DAF’s robustness to sampling noise by varying the Sensitivity Batch Num  $M$  (the number of batches used to estimate  $s_p^{(t)}$ ). We swept  $M$  across  $\{8, 12, 16, 20, 28, 32\}$ , with  $M = 16$  being the default. As detailed in Appendix H, our results demonstrate high robustness: the mean accuracy on VTAB-1k fluctuates negligibly (within  $\approx 0.3\%$ ) across this wide range. Furthermore, loss curves on CIFAR-100 confirm that training convergence remains smooth and consistent, validating that our context-aware analysis captures stable macro-level trends even with varying sample sizes.

**Comparison of Sensitivity Criteria.** We further examine

the choice of the sensitivity metric used in DAF. Our goal is not to claim that  $|g \cdot w|$  is a unique theoretical measure of future adaptation potential, but to use it as a practical first-order proxy for immediate adaptation utility under the current model state. From a first-order Taylor perspective, the local loss change satisfies  $\Delta \mathcal{L} \approx g \cdot \Delta w$ . While pruning-based criteria often use related scores to estimate the effect of removing parameters, DAF uses this signal in the opposite direction: to identify frozen components whose activation is likely to provide larger local loss reduction in the next reconfiguration stage. Compared with the gradient magnitude  $|g|$ , the Taylor-based score  $|g \cdot w|$  additionally accounts for parameter scale. As shown in Table 7, the proposed criterion achieves the best mean accuracy on VTAB-1k, outperforming random selection, gradient magnitude, and the Fisher-style diagonal proxy  $g^2$ . These results support  $|g \cdot w|$  as an effective lightweight first-order signal for repeated online reconfiguration, while avoiding the much higher cost of more expensive Fisher- or Hessian-style alternatives.

Table 7. Comparison of different sensitivity criteria on VTAB-1k using ViT-B/16.

| Criterion                   | Proxy         | Mean Acc. (%) |
|-----------------------------|---------------|---------------|
| Random                      | Uniform       | 73.5          |
| Gradient magnitude          | $ g $         | 75.0          |
| Fisher-style diagonal proxy | $g^2$         | 75.4          |
| Taylor-based (DAF)          | $ g \cdot w $ | <b>76.4</b>   |

## 5. Conclusion

In the paper, we identify a fundamental limitation in current PEFT methods: the static allocation paradigm, where the fine-tuning structure remains immutable despite the model’s evolving optimization needs. To address this, we introduce the Dynamic Adaptive Fine-tuning (DAF) framework, which bridges the gap between fixed parameter budgets and dynamic learning states. By employing a novel context-aware decoupled sensitivity analysis, we resolve the signal noise issue in dynamic evaluation, enabling precise identification of optimization bottlenecks. This analysis synergizes with our Rebuild-and-Refocus strategy, which preserves learned knowledge in frozen modules while decisively reallocating resources to critical new regions. Crucially, the mechanism maximizes adaptivity while maintaining the stability of learned representations. Extensive experiments demonstrate that DAF significantly outperforms mainstream static baselines, achieving superior performance across challenging vision benchmarks. Our work proposes a new paradigm for PEFT: shifting away from static, predefined structures to embracing a dynamic plasticity where the model’s topology evolves in tandem with its weights. Future work will extend this principle to multimodal domains and explore synergies with other efficiency techniques.

## Acknowledgements

This work was supported in part by Special Fund for Research on National Major Research Instruments of National Natural Science Foundation of China under Grant 62527806, and the National Natural Science Foundation of China under Grants 62495091, 62125305, and 52508037.

## Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

## References

- Beattie, C., Leibo, J. Z., Teplyashin, D., Ward, T., Wainwright, M., Küttler, H., Lefrancq, A., Green, S., Valdés, V., Sadik, A., et al. Deepmind lab. *arXiv preprint arXiv:1612.03801*, 2016.
- Cai, Z. and Vasconcelos, N. Cascade r-cnn: Delving into high quality object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 6154–6162, 2018.
- Chavan, A., Liu, Z., Gupta, D., Xing, E., and Shen, Z. One-for-all: Generalized lora for parameter-efficient fine-tuning. *arXiv preprint arXiv:2306.07967*, 2023.
- Chen, S., Ge, C., Tong, Z., Wang, J., Song, Y., Wang, J., and Luo, P. Adaptformer: Adapting vision transformers for scalable visual recognition. *Advances in Neural Information Processing Systems*, 35:16664–16678, 2022.
- Cimpoi, M., Maji, S., Kokkinos, I., Mohamed, S., and Vedaldi, A. Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3606–3613, 2014.
- Cuadros, J. and Bresnick, G. Eyepacs: an adaptable telemedicine system for diabetic retinopathy screening. *Journal of diabetes science and technology*, 3(3):509–516, 2009.
- Dimitrovski, I., Spasev, V., and Kitanovski, I. Few-shot remote sensing image scene classification with clip and prompt learning. *Journal of Intelligent Information Systems*, pp. 1–36, 2025.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 7:5, 2020.
- Evci, U., Dumoulin, V., Larochelle, H., and Mozer, M. C. Head2toe: Utilizing intermediate representations for better transfer learning. In *International Conference on Machine Learning*, pp. 6009–6033. PMLR, 2022.
- Fei-Fei, L., Fergus, R., and Perona, P. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *2004 conference on computer vision and pattern recognition workshop*, pp. 178–178. IEEE, 2004.
- Geiger, A., Lenz, P., Stiller, C., and Urtasun, R. Vision meets robotics: The kitti dataset. *The international journal of robotics research*, 32(11):1231–1237, 2013.
- He, H., Cai, J., Zhang, J., Tao, D., and Zhuang, B. Sensitivity-aware visual parameter-efficient fine-tuning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11825–11835, 2023.
- He, K., Gkioxari, G., Dollár, P., and Girshick, R. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pp. 2961–2969, 2017.
- Helber, P., Bischke, B., Dengel, A., and Borth, D. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019.
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. beta-vae: Learning basic visual concepts with a constrained variational framework. In *International conference on learning representations*, 2017.
- Hill, B. A. Structural plasticity as active inference: A biologically-inspired architecture for homeostatic control. *arXiv preprint arXiv:2511.02241*, 2025.
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pp. 2790–2799. PMLR, 2019.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Jia, M., Tang, L., Chen, B.-C., Cardie, C., Belongie, S., Hariharan, B., and Lim, S.-N. Visual prompt tuning. In *European conference on computer vision*, pp. 709–727. Springer, 2022.
- Jiang, Z., Mao, C., Huang, Z., Ma, A., Lv, Y., Shen, Y., Zhao, D., and Zhou, J. Res-tuning: A flexible and efficient tuning paradigm via unbinding tuner from backbone.

- Advances in Neural Information Processing Systems*, 36: 42689–42716, 2023.
- Jie, S., Wang, H., and Deng, Z.-H. Revisiting the parameter efficiency of adapters from the perspective of precision redundancy. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 17217–17226, 2023.
- Johnson, J., Hariharan, B., Van Der Maaten, L., Fei-Fei, L., Lawrence Zitnick, C., and Girshick, R. Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2901–2910, 2017.
- Khosla, A., Jayadevaprakash, N., Yao, B., and Li, F.-F. Novel dataset for fine-grained image categorization: Stanford dogs. In *Proc. CVPR workshop on fine-grained visual categorization (FGVC)*, volume 2, 2011.
- Krause, J., Stark, M., Deng, J., and Fei-Fei, L. 3d object representations for fine-grained categorization. In *Proceedings of the IEEE international conference on computer vision workshops*, pp. 554–561, 2013.
- Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. 2009.
- LeCun, Y., Huang, F. J., and Bottou, L. Learning methods for generic object recognition with invariance to pose and lighting. In *Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2004. CVPR 2004.*, volume 2, pp. II–104. IEEE, 2004.
- Lian, D., Zhou, D., Feng, J., and Wang, X. Scaling & shifting your features: A new baseline for efficient model tuning. *Advances in Neural Information Processing Systems*, 35:109–123, 2022.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft coco: Common objects in context. In *European conference on computer vision*, pp. 740–755. Springer, 2014.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., and Chen, M.-H. Dora: Weight-decomposed low-rank adaptation. In *Forty-first International Conference on Machine Learning*, 2024a.
- Liu, T., Liu, X., Shi, L., Xu, Z., Hu, Y., Huang, S., Xin, Y., Zhong, B., and Wang, D. Sparse-tuning: Adapting vision transformers with efficient fine-tuning and inference. *arXiv preprint arXiv:2405.14700*, 2024b.
- Liu, X., Zhang, F., Wu, Y., Jia, X., Xia, Z., Zhuang, F., Zhang, Z., Jiang, F., and Lin, W. Cat-id2: Category-tree integrated document identifier learning for generative retrieval in e-commerce. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, pp. 426–435, 2026.
- Loshchilov, I. and Hutter, F. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.
- Ma, G., Li, L., Wang, Z., Shen, L., Bacon, P.-L., and Tao, D. Network sparsity unlocks the scaling potential of deep reinforcement learning. *arXiv preprint arXiv:2506.17204*, 2025.
- Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A. Y., et al. Reading digits in natural images with unsupervised feature learning. In *NIPS workshop on deep learning and unsupervised feature learning*, volume 2011, pp. 7. Granada, 2011.
- Nilsback, M.-E. and Zisserman, A. Automated flower classification over a large number of classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics Image Processing*, pp. 722–729, 2008. doi: 10.1109/ICVGIP.2008.47.
- Parkhi, O. M., Vedaldi, A., Zisserman, A., and Jawahar, C. V. Cats and dogs. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3498–3505, 2012. doi: 10.1109/CVPR.2012.6248092.
- Payeur, A., Guerguiev, J., Zenke, F., Richards, B. A., and Naud, R. Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits. *Nature neuroscience*, 24(7):1010–1019, 2021.
- Poo, M.-m., Pignatelli, M., Ryan, T. J., Tonegawa, S., Bonhoeffer, T., Martin, K. C., Rudenko, A., Tsai, L.-H., Tsien, R. W., Fishell, G., et al. What is memory? the present state of the engram. *BMC biology*, 14(1):40, 2016.
- Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., and Hsieh, C.-J. Dynamicvit: Efficient vision transformers with dynamic token sparsification. *Advances in neural information processing systems*, 34:13937–13949, 2021.
- Smith, L. N. Cyclical learning rates for training neural networks. In *2017 IEEE winter conference on applications of computer vision (WACV)*, pp. 464–472. IEEE, 2017.
- Steitz, J.-M. O. and Roth, S. Adapters strike back. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 23449–23459, 2024.
- Sung, Y.-L., Cho, J., and Bansal, M. Lst: Ladder side-tuning for parameter and memory efficient transfer learning. *Advances in Neural Information Processing Systems*, 35: 12991–13005, 2022.

- Tu, C.-H., Mai, Z., and Chao, W.-L. Visual query tuning: Towards effective usage of intermediate representations for parameter and memory efficient transfer learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 7725–7735, 2023.
- Van Horn, G., Branson, S., Farrell, R., Haber, S., Barry, J., Ipeirotis, P., Perona, P., and Belongie, S. Building a bird recognition app and large scale dataset with citizen scientists: The fine print in fine-grained dataset collection. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 595–604, 2015. doi: 10.1109/CVPR.2015.7298658.
- Veeling, B. S., Linmans, J., Winkens, J., Cohen, T., and Welling, M. Rotation equivariant cnns for digital pathology. In *International Conference on Medical image computing and computer-assisted intervention*, pp. 210–218. Springer, 2018.
- Veeramacheneni, L., Wolter, M., Kuehne, H., and Gall, J. Canonical rank adaptation: An efficient fine-tuning strategy for vision transformers. In *Forty-second International Conference on Machine Learning*, 2025.
- Wah, C., Branson, S., Welinder, P., Perona, P., and Belongie, S. The caltech-ucsd birds-200-2011 dataset. 2011.
- Wang, J., Yang, G., Chen, W., Yi, H., Wu, X., Lin, Z., and Lao, Q. Mlae: Masked lora experts for visual parameter-efficient fine-tuning. *arXiv preprint arXiv:2405.18897*, 2024.
- Wang, J., Fan, L., Zhang, D., Jing, W., Di, D., Song, Y., Liu, S., and Cong, C. Visual prompt-agnostic evolution. *arXiv preprint arXiv:2601.20232*, 2026a.
- Wang, L., Xie, J., Zhang, X., Huang, M., Su, H., and Zhu, J. Hierarchical decomposition of prompt-based continual learning: Rethinking obscured sub-optimality. *Advances in Neural Information Processing Systems*, 36:69054–69076, 2023.
- Wang, Y., Huang, R., Song, S., Huang, Z., and Huang, G. Not all images are worth 16x16 words: Dynamic transformers for efficient image recognition. *Advances in neural information processing systems*, 34:11960–11973, 2021.
- Wang, Y., Jiang, X., Cheng, D., Zhao, X., Wang, Z., Li, D., and Zhao, C. Exploring interpretability for visual prompt tuning with cross-layer concepts. In *The Fourteenth International Conference on Learning Representations*, 2026b.
- Wu, Y., Piao, H., Huang, L.-K., Wang, R., Li, W., Pfister, H., Meng, D., Ma, K., and Wei, Y. Sd-lora: Scalable decoupled low-rank adaptation for class incremental learning. *arXiv preprint arXiv:2501.13198*, 2025.
- Xiao, J., Hays, J., Ehinger, K. A., Oliva, A., and Torralba, A. Sun database: Large-scale scene recognition from abbey to zoo. In *2010 IEEE computer society conference on computer vision and pattern recognition*, pp. 3485–3492. IEEE, 2010.
- Xiao, T., Liu, Y., Zhou, B., Jiang, Y., and Sun, J. Unified perceptual parsing for scene understanding. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 418–434, 2018.
- Xiong, Y., Chen, H., Hao, T., Lin, Z., Han, J., Zhang, Y., Wang, G., Bao, Y., and Ding, G. Pyra: Parallel yielding reactivation for training-inference efficient task adaptation. In *European Conference on Computer Vision*, pp. 455–473. Springer, 2024.
- Yin, D., Yang, Y., Wang, Z., Yu, H., Wei, K., and Sun, X. 1% vs 100%: Parameter-efficient low rank adapter for dense predictions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 20116–20126, 2023.
- Yin, D., Han, X., Li, B., Feng, H., and Bai, J. Parameter-efficient is not sufficient: Exploring parameter, memory, and time efficient adapter tuning for dense predictions. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 1398–1406, 2024.
- Yin, D., Hu, L., Li, B., Zhang, Y., and Yang, X. 5%>100%: Breaking performance shackles of full fine-tuning on visual recognition tasks. In *CVPR*, pp. 20071–20081. Computer Vision Foundation / IEEE, 2025.
- Zaken, E. B., Goldberg, Y., and Ravfogel, S. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 1–9, 2022.
- Zhai, X., Puigcerver, J., Kolesnikov, A., Ruysen, P., Riquelme, C., Lucic, M., Djolonga, J., Pinto, A. S., Neumann, M., Dosovitskiy, A., et al. A large-scale study of representation learning with the visual task adaptation benchmark. *arXiv preprint arXiv:1910.04867*, 2019.
- Zhai, X., Kolesnikov, A., Houlsby, N., and Beyer, L. Scaling vision transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 12104–12113, 2022.
- Zhang, F., Liu, X., Jia, X., Zhang, Y., Xia, Z., Jiang, F., Zhuang, F., Lin, W., and Zhang, Z. Hiergr: Hierarchical semantic representation enhancement for generative retrieval in food delivery search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 444–455, 2025a.

- Zhang, F., Liu, X., Jia, X., Zhang, Y., Zhang, S., Li, X., Zhuang, F., Lin, W., and Zhang, Z. Multi-level relevance document identifier learning for generative retrieval. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 10066–10080, 2025b.
- Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W., and Zhao, T. Adaptive budget allocation for parameter-efficient fine-tuning. In *The Eleventh International Conference on Learning Representations*, 2023.
- Zhang, T., Bai, J., Lu, Z., Lian, D., Wang, G., Wang, X., and Xia, S.-T. Parameter-efficient and memory-efficient tuning for vision transformer: a disentangled approach. In *European Conference on Computer Vision*, pp. 346–363. Springer, 2024a.
- Zhang, Y., Zhou, K., and Liu, Z. Neural prompt search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024b.
- Zhang, Z., Zhang, Q., Gao, Z., Zhang, R., Shutova, E., Zhou, S., and Zhang, S. Gradient-based parameter selection for efficient fine-tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 28566–28577, 2024c.
- Zhao, W., Tang, J., Han, Y., Song, Y., Wang, K., Huang, G., Wang, F., and You, Y. Dynamic tuning towards parameter and inference efficiency for vit adaptation. *Advances in Neural Information Processing Systems*, 37:114765–114796, 2024.
- Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A., and Torralba, A. Scene parsing through ade20k dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 633–641, 2017.
- Zhou, N., Chen, J., and Huang, D. Sharing task-relevant information in visual prompt tuning by cross-layer dynamic connection. *IEEE Transactions on Image Processing*, 2025.

## A. Dataset Details

The experiments are conducted on two diverse and challenging benchmarks: the Fine-Grained Visual Classification (FGVC) suite and the Visual Task Adaptation Benchmark (VTAB-1k) (Zhai et al., 2019).

The VTAB-1k benchmark consists of 19 distinct tasks as shown in Table 8, which are grouped into three categories. The Natural category includes common image classification datasets such as CIFAR-100 (Krizhevsky et al., 2009), Caltech101 (Fei-Fei et al., 2004), DTD (Cimpoi et al., 2014), Oxford-Flowers102 (Nilsback & Zisserman, 2008), Oxford-Pets (Parkhi et al., 2012), SVHN (Netzer et al., 2011), and Sun397 (Xiao et al., 2010). The Specialized category is composed of tasks from specific domains, including medical imaging (Patch Camelyon (Veeling et al., 2018), Retinopathy (Cuadros & Bresnick, 2009)) and satellite imagery (EuroSAT (Helber et al., 2019), Resisc45 (Dimitrovski et al., 2025)). The Structured category evaluates the model’s understanding of scenes and semantics, with tasks like Clevr (Johnson et al., 2017), DMLab (Beattie et al., 2016), KITTI-Dist (Geiger et al., 2013), dSprites (Higgins et al., 2017), and SmallNORB (LeCun et al., 2004). For all VTAB-1k tasks, we follow the standard setup of using 800 training and 200 validation samples.

The FGVC benchmark focuses on tasks that require distinguishing between subtle visual differences. We evaluate on five datasets from this benchmark: CUB-200-2011 (Wah et al., 2011), NABirds (Van Horn et al., 2015), Oxford-Flowers102 (Nilsback & Zisserman, 2008), Stanford Cars (Krause et al., 2013), and Stanford Dogs (Khosla et al., 2011). For these datasets, we use the official training, validation, and test splits provided by the dataset creators.

Table 8. Statistics of the datasets used in the experiments. For VTAB-1k, all tasks use 800 training and 200 validation samples. For FGVC, we list the official split sizes.

| Benchmark          | Dataset              | # Classes | Train  | Val   | Test   |
|--------------------|----------------------|-----------|--------|-------|--------|
| VTAB-1k            |                      |           |        |       |        |
| <i>Natural</i>     | CIFAR100             | 100       | 800    | 200   | 10,000 |
|                    | Caltech101           | 102       | 800    | 200   | 6,084  |
|                    | DTD                  | 47        | 800    | 200   | 1,880  |
|                    | Oxford-Flowers102    | 102       | 800    | 200   | 6,149  |
|                    | Oxford-Pets          | 37        | 800    | 200   | 3,669  |
|                    | SVHN                 | 10        | 800    | 200   | 26,032 |
|                    | Sun397               | 397       | 800    | 200   | 21,750 |
| <i>Specialized</i> | Patch Camelyon       | 2         | 800    | 200   | 32,768 |
|                    | EuroSAT              | 10        | 800    | 200   | 5,400  |
|                    | Resisc45             | 45        | 800    | 200   | 6,300  |
|                    | Retinopathy          | 5         | 800    | 200   | 42,670 |
| <i>Structured</i>  | Clevr/count          | 8         | 800    | 200   | 15,000 |
|                    | Clevr/distance       | 6         | 800    | 200   | 15,000 |
|                    | DMLab                | 6         | 800    | 200   | 22,735 |
|                    | KITTI-Dist           | 4         | 800    | 200   | 711    |
|                    | dSprites/location    | 16        | 800    | 200   | 73,728 |
|                    | dSprites/orientation | 16        | 800    | 200   | 73,728 |
|                    | SmallNORB/azimuth    | 18        | 800    | 200   | 12,150 |
|                    | SmallNORB/elevation  | 18        | 800    | 200   | 12,150 |
| FGVC               |                      |           |        |       |        |
| <i>FGVC</i>        | CUB-200-2011         | 200       | 5,994  | –     | 5,794  |
|                    | NABirds              | 555       | 23,929 | –     | 24,633 |
|                    | Oxford-Flowers102    | 102       | 1,020  | 1,020 | 6,149  |
|                    | Stanford Cars        | 196       | 8,144  | –     | 8,041  |
|                    | Stanford Dogs        | 120       | 12,000 | –     | 8,580  |

## B. Setting Details

We summarize the main experimental settings used for DAF in Table 9. Unless otherwise specified, we use a unified configuration across the VTAB-1k tasks to avoid per-task tuning and to evaluate the robustness of the proposed dynamic reconfiguration strategy. For baseline methods, we follow the settings reported in their original papers or standard public implementations whenever applicable.

Table 9. General hyperparameters.

| Hyperparameter                                    | Value              |
|---------------------------------------------------|--------------------|
| <i>Optimizer</i>                                  |                    |
| Optimizer                                         | AdamW              |
| Betas                                             | (0.9, 0.999)       |
| Epsilon                                           | $1 \times 10^{-8}$ |
| <i>Training Schedule</i>                          |                    |
| Base Learning Rate                                | $3 \times 10^{-4}$ |
| Weight Decay                                      | $1 \times 10^{-4}$ |
| Learning Rate Schedule                            | Cosine Decay       |
| Warmup Epochs                                     | 10                 |
| Batch Size                                        | 64                 |
| Total Epochs                                      | 200                |
| <i>Regularization</i>                             |                    |
| Label Smoothing                                   | 0.1                |
| Drop Path Rate                                    | 0.1                |
| LoRA Rank ( $r$ )                                 | 8                  |
| <i>DAF Specific</i>                               |                    |
| Dynamic Update Interval ( $E_{\text{interval}}$ ) | 10                 |
| Parameter Budget ( $\tau$ )                       | 0.2                |
| Sensitivity Analysis Batch Size                   | 16                 |

### C. Visual Analysis of Training Stability

To empirically address the concern regarding training stability due to optimizer state re-initialization, we visualized the training loss trajectories over 200 epochs on four representative datasets: CIFAR-100, Caltech-101, DTD, and Flower102.

As illustrated in Figure 4, we observe three critical behaviors that validate the robustness of the dynamic reconfiguration paradigm:

- **Rapid Convergence via Sensitivity Guidance.** Across all datasets, DAF (Red line) exhibits the fastest convergence rate. Despite the momentary loss of momentum during structural updates, the model quickly recovers and accelerates. This effectively demonstrates that dynamically reallocating the parameter budget to high-sensitivity regions provides a stronger gradient signal than maintaining a static, sub-optimal topology.
- **Topology-Induced Exploration via Optimizer Reset.** The loss curves exhibit characteristic periodic fluctuations triggered by the optimizer reset. Although this momentarily increases loss, we argue that this operation is theoretically grounded in the non-stationary nature of our dynamic training. As DAF reconfigures the trainable topology, the underlying optimization landscape shifts, rendering historical momentum stale. Recent work on non-stationary optimization (Ma et al., 2025) indicates that resetting optimizer states is crucial to eliminate bias from outdated gradient histories. Furthermore, these periodic perturbations function similarly to Stochastic Gradient Descent with Warm Restarts (SGDR) (Loshchilov & Hutter, 2017) or Cyclical Learning Rates (Smith, 2017). As seen in the curves, the transient spikes are consistently followed by steep drops in loss, confirming that shedding historical momentum effectively helps the model escape sharp local minima or saddle points to discover better optimization paths.
- **Superior Asymptotic Performance.** Crucially, DAF consistently settles at a significantly lower final loss level compared to both Static DAF and LoRA (post-150 epochs). On CIFAR-100, the pronounced gap in training loss aligns directly with the substantial accuracy improvement (74.1% vs 68.1%), confirming that the dynamic topology facilitates a better fit to the underlying data manifold.

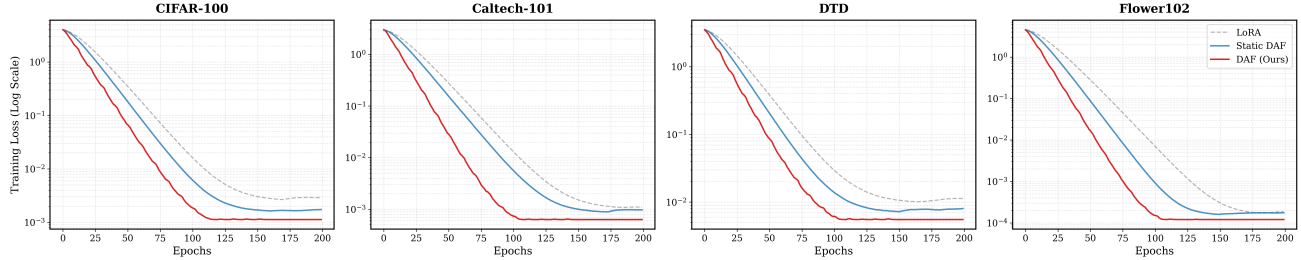


Figure 4. Training Loss Trajectories. Comparison of DAF (Ours), Static DAF, and LoRA. DAF consistently achieves the fastest convergence and the lowest final loss. Note that the periodic fluctuations correspond to the reconfiguration interval ( $E_{interval} = 10$ ), serving as beneficial structural perturbations that prevent stagnation in sub-optimal local minima. (Curves are smoothed via exponential moving average for visual clarity.)

## D. Detailed Per-Task Results

Here we provide a more granular analysis with detailed per-task results on both the FGVC and VTAB-1k benchmarks, presented in Table 10 and Table 11 respectively. In the fine-grained tasks of FGVC (Table 10), where identifying subtle visual cues is paramount, DAF’s ability to shift its focus during training allows it to capture a wider range of discriminative features. On the diverse tasks of VTAB-1k (Table 11), DAF shows notable strength, particularly on the Structured category. These tasks (e.g., DMLab, KITTI-Dist) exhibit a large domain shift from the pre-training data, a scenario where static methods often falter. DAF’s adaptability allows it to better navigate these challenging domain gaps, leading to superior robustness.

Table 10. Detailed per-task Top-1 accuracy (%) on the FGVC benchmark with ViT-B/16 (ImageNet-21k pre-trained).

| Method                              | CUB-200-2011 | NABirds     | OxfordFlowers | StanfordDogs | StanfordCars | Mean Acc. (%) |
|-------------------------------------|--------------|-------------|---------------|--------------|--------------|---------------|
| Full Fine-tuning                    | 87.3         | 82.7        | 98.8          | 89.4         | 84.5         | 88.5          |
| <i>Static PEFT Baselines</i>        |              |             |               |              |              |               |
| Adapter-8                           | 87.3         | <b>84.3</b> | 98.4          | 88.8         | 68.4         | 85.5          |
| Adapter-32                          | 87.2         | <b>84.3</b> | 98.5          | 89.6         | 68.4         | 85.6          |
| LoRA-8                              | 84.9         | 79.0        | 98.1          | 88.1         | 79.8         | 86.0          |
| LoRA-16                             | 85.6         | 79.8        | 98.9          | 87.6         | 72.0         | 84.8          |
| VPT-Deep                            | 84.5         | 78.5        | 98.5          | 86.5         | 71.0         | 83.8          |
| AdaptFormer                         | 87.0         | 83.2        | 98.8          | 89.0         | 72.5         | 86.1          |
| NOAH                                | 88.6         | 83.3        | <u>99.3</u>   | 90.8         | 84.0         | 89.2          |
| <i>Recent Static SOTA Baselines</i> |              |             |               |              |              |               |
| VQT                                 | 83.0         | 77.0        | 97.5          | 85.0         | 70.0         | 82.5          |
| SPT-LoRA                            | 88.6         | 82.8        | <b>99.4</b>   | <u>91.4</u>  | 84.5         | 89.3          |
| SPT-Adapter                         | <b>89.1</b>  | 83.3        | 99.2          | 90.5         | 85.6         | 89.5          |
| Res-Tuning                          | <u>88.7</u>  | 83.8        | <u>99.3</u>   | 91.2         | <u>87.3</u>  | 90.0          |
| Bi-LoRA                             | 87.8         | 83.5        | 99.0          | 90.2         | 86.0         | 89.3          |
| SynQT                               | 85.5         | 79.5        | 98.7          | 87.8         | 72.0         | 84.7          |
| AdaLoRA                             | 88.5         | 82.5        | 99.1          | 91.2         | 85.9         | 89.4          |
| PYRA                                | 86.8         | 82.5        | 98.8          | 89.1         | 73.8         | 86.2          |
| GPS                                 | 88.6         | 83.6        | <u>99.3</u>   | 91.3         | 87.2         | 90.0          |
| Adapter+(r=1)                       | 88.6         | <u>83.9</u> | <u>99.3</u>   | <u>91.4</u>  | <u>87.3</u>  | <u>90.1</u>   |
| DyT(r=0.5)                          | 88.5         | 83.8        | <u>99.3</u>   | 91.3         | 87.1         | 90.0          |
| <i>Our Methods</i>                  |              |             |               |              |              |               |
| Static DAF (Ours)                   | 88.2         | 82.5        | 99.0          | 91.0         | 86.8         | 89.5          |
| <b>DAF (Ours)</b>                   | <u>88.7</u>  | <u>83.9</u> | <b>99.4</b>   | <b>91.5</b>  | <b>87.4</b>  | <b>90.2</b>   |

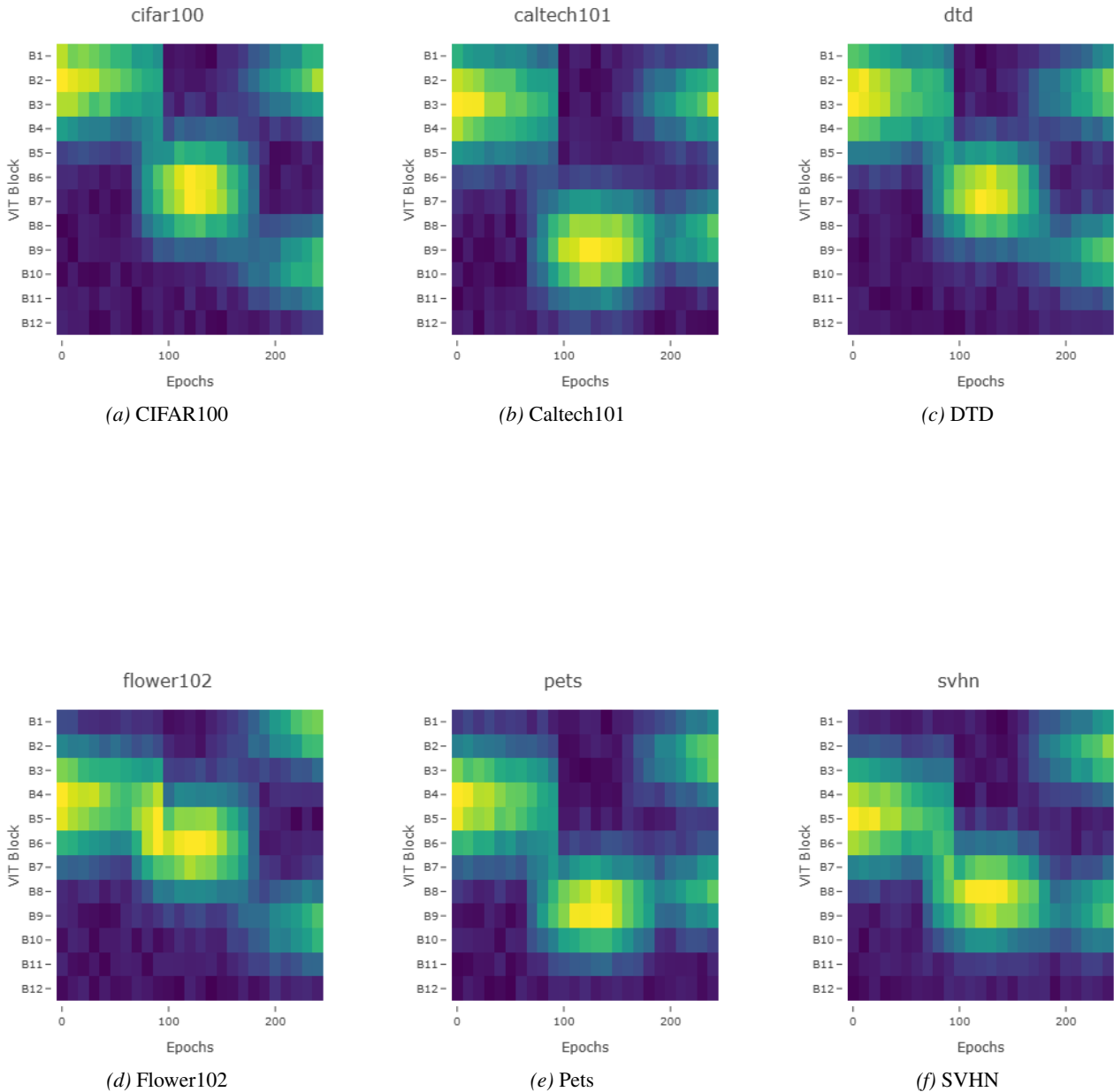
Table 11. Detailed per-task Top-1 accuracy (%) on the VTAB-1k benchmark with ViT-B/16 (ImageNet-21k pre-trained). Note the performance across Natural, Specialized, and Structured task categories.

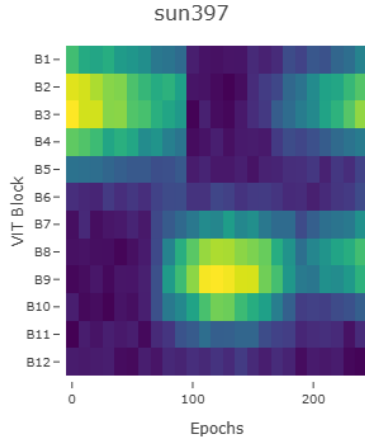
| Method                              | Natural     |             |             |             |             |             |             | Specialized |             |             |             | Structured  |             |             |             |             |             |             |             | Mean Acc. (%) |
|-------------------------------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|---------------|
|                                     | CIFAR100    | Caltech101  | DTD         | Flower102   | Pets        | SVHN        | Sun397      | EuroSAT     | Resisc45    | Retinopathy | Camelyon    | Clevr-Count | Clevr-Dist  | DMLab       | KITTI-Dist  | dSpr-Loc    | dSpr-Ori    | sNORB-Azim  | sNORB-Ele   |               |
| Full Fine-tuning                    | 68.9        | 87.7        | 64.3        | 97.2        | 86.9        | 87.4        | 38.8        | 95.7        | 84.2        | 73.9        | 79.7        | 56.3        | 58.6        | 41.7        | 65.5        | 57.5        | 46.7        | 25.7        | 29.1        | 69.0          |
| <i>Static PEFT Baselines</i>        |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |               |
| Adapter-32                          | 68.7        | 92.2        | 69.8        | 98.9        | 90.3        | 84.2        | 53.0        | 95.4        | 83.2        | 74.3        | 83.2        | 81.9        | 63.9        | 48.7        | 80.6        | 76.2        | 47.6        | 30.8        | 36.4        | 74.0          |
| LoRA-16                             | 68.1        | 91.4        | 69.8        | 99.0        | 90.5        | 86.4        | 53.1        | 95.8        | 84.7        | 74.2        | 85.1        | 83.0        | 66.9        | 50.4        | 81.4        | 80.2        | 46.6        | 32.2        | 41.1        | 75.0          |
| VPT-Deep                            | 78.8        | 90.8        | 65.8        | 98.0        | 88.3        | 78.1        | 49.6        | 96.1        | 83.4        | 68.4        | 81.8        | 68.5        | 60.0        | 46.5        | 72.8        | 73.6        | 47.9        | <u>32.9</u> | 37.8        | 72.0          |
| AdaptFormer                         | 70.8        | 90.8        | 70.5        | 99.1        | 90.9        | 86.6        | 54.8        | 95.8        | 84.4        | <b>76.3</b> | 83.0        | 81.9        | 64.3        | 49.3        | 80.3        | 76.3        | 45.7        | 31.7        | 41.1        | 74.7          |
| NOAH                                | 69.6        | 92.7        | 70.2        | 99.1        | 90.4        | 86.1        | 53.7        | 95.4        | 83.9        | 75.8        | 84.4        | 82.8        | <b>68.9</b> | 49.9        | 81.7        | 81.8        | 48.3        | 32.8        | 44.2        | 75.5          |
| <i>Recent Static SOTA Baselines</i> |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |               |
| VQT                                 | 66.3        | 89.9        | 67.8        | 97.9        | 84.7        | 79.9        | 45.5        | 95.2        | 80.9        | 74.7        | 79.0        | 46.7        | 61.6        | 45.1        | 63.6        | 62.9        | 32.1        | 30.0        | 28.8        | 68.3          |
| SPT-Adapter                         | 72.9        | <u>93.2</u> | 72.5        | <u>99.3</u> | 91.4        | 84.6        | 55.2        | 96.0        | 84.3        | 75.5        | 85.3        | 82.2        | <u>68.0</u> | 49.3        | 80.0        | 82.4        | 51.9        | 31.7        | 41.2        | 75.8          |
| SPT-LoRA                            | 72.3        | 93.0        | 72.5        | <u>99.3</u> | 91.5        | 86.2        | 55.5        | 96.2        | 85.1        | <u>75.9</u> | 85.0        | 83.7        | 66.4        | <u>52.5</u> | 80.2        | 80.1        | 51.1        | 30.1        | 41.3        | 75.9          |
| Res-Tuning                          | 75.2        | 92.7        | 71.9        | <u>99.3</u> | <b>91.9</b> | 86.7        | <b>58.5</b> | 95.6        | 85.0        | 74.6        | 86.7        | 80.2        | 63.6        | 50.6        | 80.2        | <b>85.4</b> | <b>55.7</b> | 31.9        | 42.0        | 76.2          |
| Bi-LoRA                             | 72.6        | 90.4        | 71.8        | 99.0        | 91.3        | 87.0        | 56.0        | 94.1        | 82.1        | 75.4        | 86.1        | 81.0        | 64.2        | 50.5        | 79.7        | 83.0        | 53.7        | 29.7        | 42.9        | 75.3          |
| SynQT                               | 70.9        | 89.7        | 68.8        | 98.5        | 89.6        | 77.8        | 50.6        | <b>96.7</b> | 83.5        | 75.2        | 82.3        | 71.8        | 62.7        | 48.5        | 75.4        | 74.1        | 49.0        | 31.7        | 36.1        | 72.9          |
| AdaLoRA                             | 74.1        | 93.1        | 72.6        | 99.1        | 91.2        | 87.0        | 55.3        | 96.1        | 84.9        | 75.8        | 84.7        | 83.7        | 66.0        | 51.7        | 81.0        | 80.5        | 50.7        | 30.0        | 40.8        | 75.9          |
| PYRA                                | 67.5        | 90.3        | 69.3        | 98.9        | 90.0        | 84.6        | 53.1        | 95.7        | 83.3        | 75.2        | 83.3        | 82.6        | <b>68.9</b> | 50.8        | 80.0        | 81.8        | 45.8        | 32.2        | 42.8        | 74.7          |
| GPS                                 | <u>79.8</u> | 92.9        | <b>74.5</b> | 98.1        | 90.4        | <b>90.3</b> | 51.1        | 94.9        | 85.2        | 75.2        | 86.6        | 78.6        | 61.3        | <b>53.7</b> | 81.1        | 82.7        | 53.1        | 29.4        | <b>44.8</b> | 76.2          |
| Adapter+(r=1)                       | <b>85.4</b> | 92.4        | <u>73.1</u> | 99.1        | 91.3        | 83.1        | <u>58.1</u> | <u>96.6</u> | <u>85.3</u> | 72.6        | <b>87.2</b> | 80.7        | 60.6        | 50.9        | 79.9        | <u>83.3</u> | <u>55.6</u> | 27.1        | 43.0        | <u>76.3</u>   |
| DyT(r=0.5)                          | 70.4        | <b>94.2</b> | 71.1        | 99.1        | <u>91.7</u> | <u>88.0</u> | 51.5        | 95.3        | 84.2        | 75.8        | <u>87.1</u> | 79.2        | 61.8        | 51.0        | <b>82.4</b> | 79.7        | 52.3        | <b>35.3</b> | <u>44.5</u> | 75.7          |
| <i>Our Methods</i>                  |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |             |               |
| Static DAF (Ours)                   | 73.4        | 92.4        | 72.8        | 99.1        | 91.0        | 86.7        | 55.0        | 95.8        | 84.4        | 75.2        | 85.3        | <u>83.9</u> | 67.3        | 51.8        | 82.0        | 80.4        | 50.1        | 30.1        | 40.9        | 75.8          |
| <b>DAF (Ours)</b>                   | 74.1        | 92.9        | 73.0        | <b>99.4</b> | <u>91.7</u> | 87.5        | 55.5        | 96.1        | <b>85.6</b> | 75.8        | 86.0        | <b>84.5</b> | 67.8        | <u>52.5</u> | <u>82.1</u> | 81.2        | 50.9        | 30.1        | 41.4        | <b>76.4</b>   |

## E. Dynamic Behavior Visualization on VTAB-1k

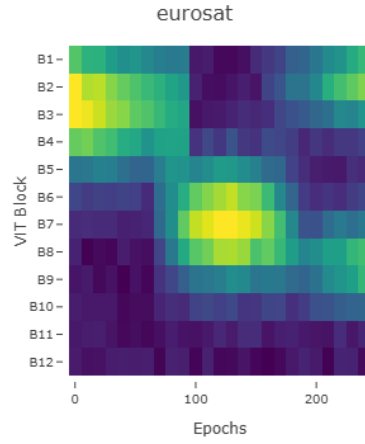
To provide a comprehensive and intuitive understanding of DAF’s working mechanism across diverse tasks, we visualize the evolution of activated LoRA modules during training for all 19 datasets in the VTAB-1k benchmark. As depicted in Figure 5, while the general pattern of shifting training is consistent, the specific layers and the timing of these shifts vary significantly from task to task.

For instance, on semantically simple tasks like EuroSAT, the model quickly identifies and focuses on a stable set of features. In contrast, on complex, structured tasks like DMLab, the activated regions exhibit a much more volatile and continuous redistribution throughout the entire training process. This adaptive behavior, which no static PEFT method can replicate, visually demonstrates how DAF intelligently and uniquely allocates its limited parametric resources for each specific task, adapting to where they are most needed as the model’s learning state evolves.

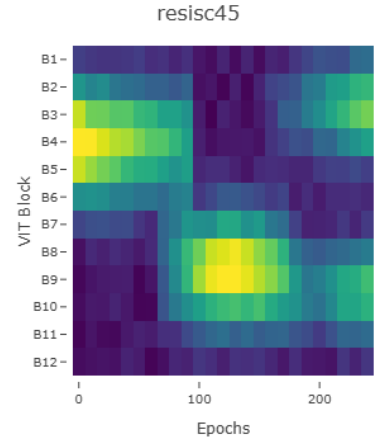




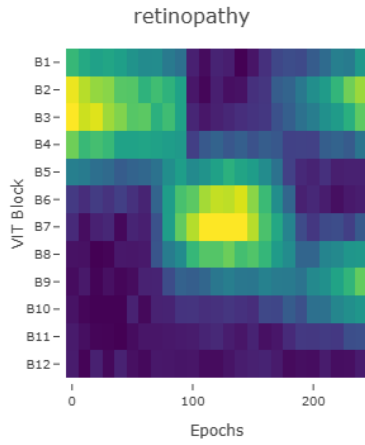
(g) Sun397



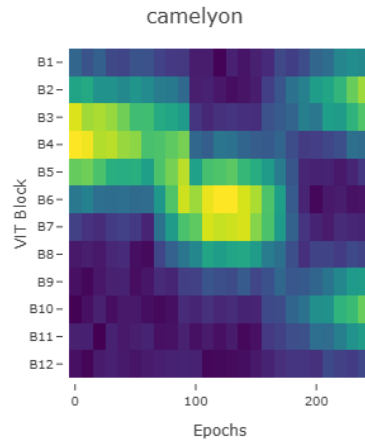
(h) EuroSAT



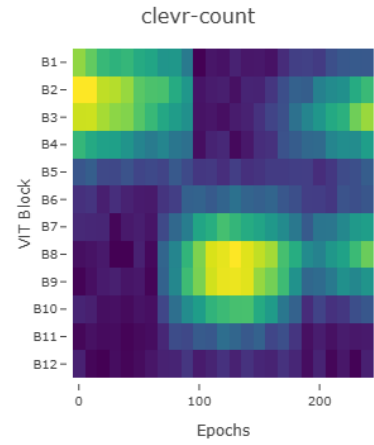
(i) Resisc45



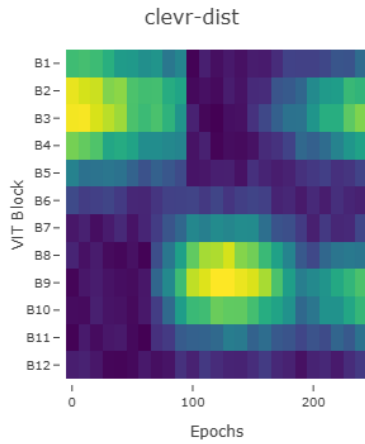
(j) Retinopathy



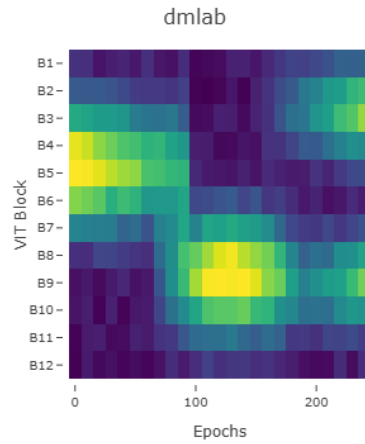
(k) Camelyon



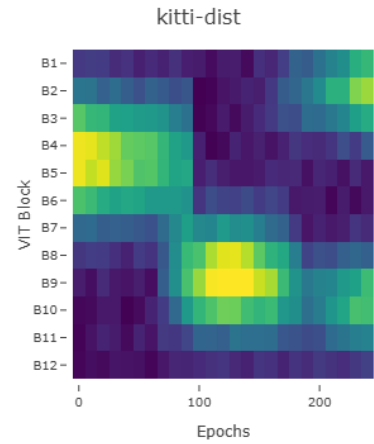
(l) Clevr-Count



(m) Clevr-Dist



(n) DMLab



(o) KITTI-Dist

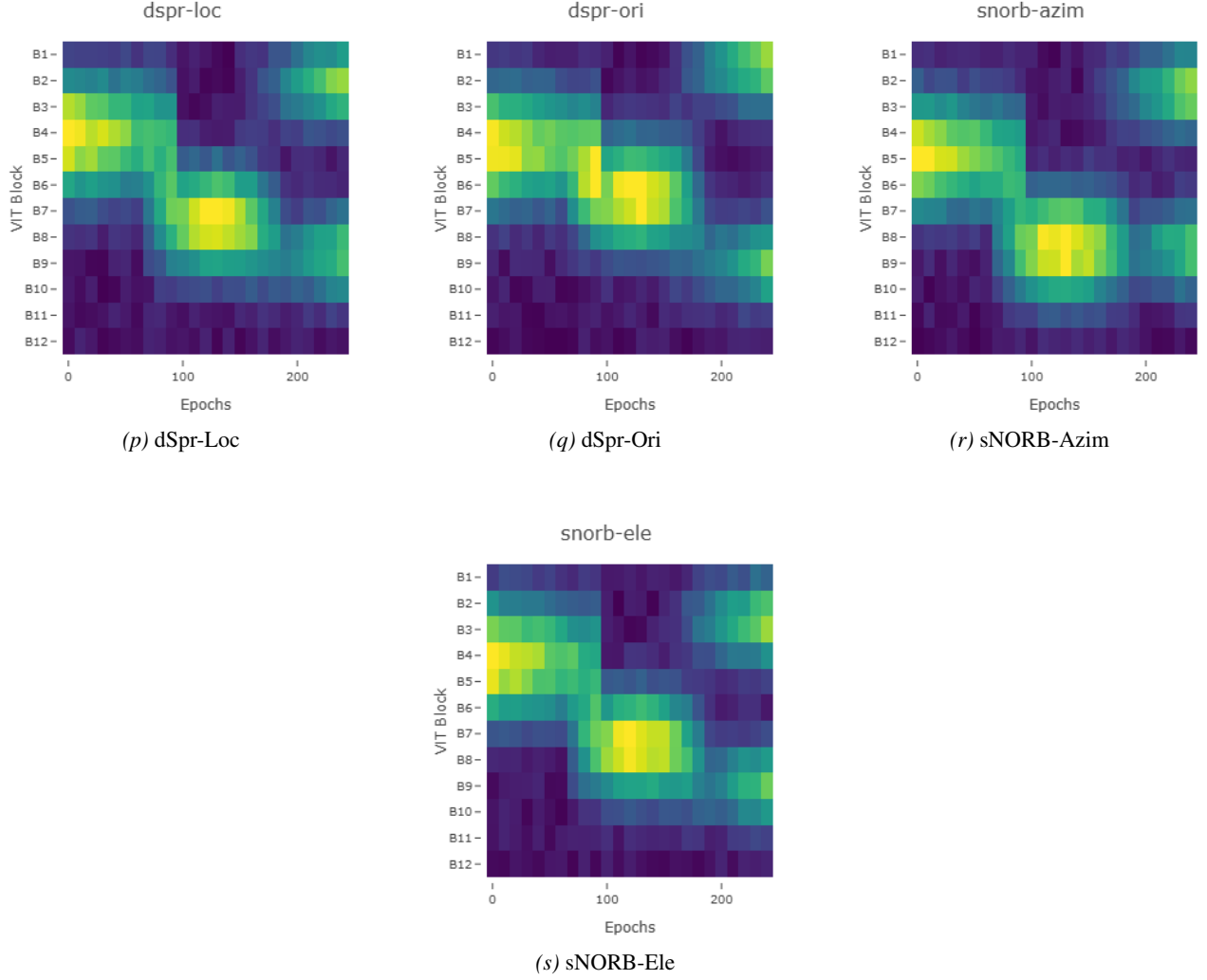


Figure 5. The dynamic evolution of activated modules by DAF across all 19 tasks in the VTAB-1k benchmark. The x-axis represents training epochs, and the y-axis represents the 12 blocks of ViT. The color intensity indicates the activation level of LoRA modules. Note the diversity in activation patterns, demonstrating DAF’s task-specific adaptability.

## F. Computational Overhead Analysis

In this subsection, we provide a comparative analysis of the computational overhead of the DAF framework against other representative PEFT methods. We focus on three key efficiency metrics: the percentage of trainable parameters, the training memory footprint, and the inference computational cost (GFLOPs). A comprehensive comparison is presented in Table 12.

**Parameter and Memory Efficiency.** As shown in Table 12, DAF is highly parameter-efficient, requiring only 0.17M trainable parameters. Furthermore, its training memory footprint of 8.64 GB is significantly lower than standard LoRA-based approaches (14.07 GB). This high efficiency stems from our decoupled training strategy. Standard fine-tuning or LoRA requires storing optimizer states (e.g., momentum and variance in AdamW) for all trainable parameters, which typically consumes 2 to 3 times the memory of the parameters themselves. In DAF, we strictly limit trainable parameters to the sparse active modules.

**Addressing Memory Consumption in Sensitivity Analysis.** A potential concern is whether computing gradients for the entire backbone during the periodic sensitivity analysis (“Perceive” phase) causes a memory spike. We clarify that this phase is mathematically guaranteed to remain memory-efficient through three specific system-level designs:

1. **No Optimizer State Instantiation:** While we compute gradients ( $\nabla_w \mathcal{L}$ ) for the backbone to measure sensitivity, we strictly do not update these weights. Consequently, no optimizer states (momentum/variance buffers) are ever allocated for the backbone, avoiding the largest memory bottleneck of full fine-tuning.
2. **Micro-Batching Strategy:** We employ a reduced micro-batch size of  $B_{micro} = 8$  solely for this analysis phase, compared to the training batch size of  $B_{train} = 64$ . Since activation memory scales linearly with batch size, this reduces the activation footprint by a factor of 8, effectively counterbalancing the cost of computing gradients for the full model.
3. **Transient Computation Graph:** Unlike training steps where the computation graph must be retained until the `optimizer.step()`, in the “Perceive” phase, gradients are used solely for calculating scalar sensitivity scores. We implement an immediate graph release mechanism where intermediate tensors are freed as soon as the metric is computed layer-by-layer.

These designs ensure that the transient memory usage during the “Perceive” phase remains strictly bounded and does not exceed the steady-state training memory reported in Table 12.

Table 12. Computational cost comparison on a ViT-B/16 backbone. Data for baseline methods are sourced from their respective publications on the VTAB-1k. DAF achieves a leading parameter efficiency while maintaining zero inference overhead.

| Method            | Params (M)  | Training Memory (GB) | Inference Memory (GB) | GFLOPs       |
|-------------------|-------------|----------------------|-----------------------|--------------|
| Full Fine-tuning  | 85.8        | 17.90                | 2.57                  | 17.58        |
| Adapter           | 1.19        | 13.74                | 2.61                  | 17.81        |
| LoRA              | 2.16        | 14.07                | 2.79                  | 17.58        |
| AdaptFormer       | 1.19        | 13.26                | 2.61                  | 17.81        |
| VPT               | 0.18        | 14.60                | 2.80                  | 18.32        |
| SPT-Adapter       | 0.35        | 13.50                | 2.61                  | 17.81        |
| SPT-LoRA          | 0.35        | 9.80                 | 2.57                  | 17.58        |
| PYRA              | 0.29        | 13.30                | 2.61                  | 17.81        |
| SynQT             | 2.73        | 3.40                 | 2.90                  | 17.20        |
| NOAH              | 0.45        | 13.50                | 2.61                  | 17.81        |
| <b>DAF (Ours)</b> | <b>0.17</b> | <b>8.64</b>          | <b>2.57</b>           | <b>17.58</b> |

**Inference Efficiency.** A key advantage of DAF is its exceptional inference efficiency. Similar to LoRA and SPT-LoRA, the learned LoRA modules in DAF can be merged into the backbone weights before deployment. This re-parameterization means that DAF introduces zero additional inference latency or computational cost (GFLOPs) compared to the original, unmodified ViT backbone. This provides a distinct advantage over methods like VPT and SynQT. Although these methods are memory-efficient during training, they require separate modules during inference that cannot be merged. This results in

increased computational cost for VPT (18.32 GFLOPs) and higher inference memory for SynQT (2.90 GB), making DAF a more streamlined solution for deployment.

In summary, DAF achieves a superior balance of parameter efficiency and deployment cost. The modest overhead introduced during the training phase by the dynamic reconfiguration mechanism is a strategic trade-off that unlocks significant performance gains (as shown in the main paper) without compromising the critical real-world deployment efficiency of the final model.

## G. The training and inference times for each module of DAF

In this section, we provide a quantitative breakdown of the computational time associated with the DAF framework, specifically focusing on the overhead introduced by the dynamic reconfiguration mechanism.

**Inference Efficiency: Zero Latency Increase.** A key design principle of DAF is to ensure efficient deployment. Although the model structure evolves during training, the final learned parameters (matrices  $A$  and  $B$ ) are mathematically merged into the backbone weights before inference:

$$W_{final} = W_0 + BA \quad (10)$$

This re-parameterization ensures that the deployed model is architecturally identical to the original ViT. Consequently, the inference time of DAF is exactly equal to that of the original pre-trained ViT model, introducing zero additional GFLOPs or latency.

**Training Efficiency: Stepwise Analysis.** To quantify the impact of the dynamic mechanism on training time, we profiled the DAF framework on the CIFAR-100 dataset using an NVIDIA RTX 4090 GPU. The training process alternates between a standard fine-tuning phase (lasting  $E_{interval} = 10$  epochs) and a dynamic update phase. The time consumption for a single update cycle (10 epochs + 1 dynamic update) is detailed in Table 13.

Table 13. Time consumption breakdown for one DAF training cycle (10 epochs) on CIFAR-100.

| Phase             | Component                         | Time Cost                                   |
|-------------------|-----------------------------------|---------------------------------------------|
| Standard Training | Fine-tuning (10 epochs)           | $8.12s \times 10$<br>$\approx$ <b>81.2s</b> |
|                   | Context-Aware Analysis (Perceive) | $\approx$ 3.12s                             |
| Dynamic Update    | Elite Selection (Decide)          | $\approx$ 0.21s                             |
|                   | Rebuild-and-Refocus (Execute)     | $\approx$ 3.34s                             |
|                   | <i>Total Dynamic Overhead</i>     | $\approx$ <b>6.67s</b>                      |

As illustrated in Table 13, the standard fine-tuning for 10 epochs consumes approximately 81.2 seconds. In contrast, the total overhead introduced by the dynamic operations sums to approximately 6.67 seconds. This represents a relative overhead of roughly 8.2% compared to the standard training time. However, this metric effectively trades computation for acceleration. As visualized in Appendix C (Figure 4), DAF converges significantly faster than static baselines. Consequently, despite the minor per-epoch computational cost, DAF achieves a superior trade-off between training efficiency and final accuracy, as it reaches the target performance ceiling with substantially fewer total iterations than static counterparts.

It is worth noting the efficiency difference between the dynamic stages. The Elite Selection (Decide) phase is extremely fast ( $\approx$  0.21s) because it primarily involves sorting scalar sensitivity scores to identify the Top- $\tau$  parameters, which has very low computational complexity. Conversely, the Context-Aware Analysis (Perceive) takes longer ( $\approx$  3.12s) as it necessitates a backward pass on the full model to compute accurate gradients. Overall, this minimal time cost validates that DAF achieves dynamic adaptability without imposing a heavy burden on training throughput. Furthermore, regarding inference, since the learned LoRA parameters are merged into the backbone weights via re-parameterization, DAF incurs zero additional computational costs during deployment, maintaining the exact same inference latency as the original ViT model.

## H. Robustness Analysis of Sensitivity Estimation

To empirically validate the robustness of our sensitivity estimation against sampling noise, we conducted a comprehensive ablation study on the Sensitivity Batch Num  $M$ . We evaluated the DAF framework by varying  $M$  from 8 to 32 on the VTAB-1k benchmark.

**Performance Stability.** As presented in Table 14, the final Mean Accuracy remains extremely stable across the entire range of  $M \in \{8, 12, 16, 20, 28, 32\}$ . The performance variance is minimal, indicating that the DAF framework is not sensitive to the exact number of batches used for sensitivity analysis, provided that a representative window is covered.

Table 14. Impact of Sensitivity Batch Num  $M$  on VTAB-1k Mean Accuracy.

| Sensitivity Batch Num $M$ | 8    | 12   | 16 (Default) | 20   | 28   | 32   |
|---------------------------|------|------|--------------|------|------|------|
| Mean Acc. (%)             | 76.3 | 76.4 | 76.4         | 76.5 | 76.4 | 76.2 |

**Training Convergence on CIFAR-100.** Furthermore, to visually assess the stability of the optimization process, we plotted the training loss curves for the CIFAR-100 task under different  $M$  settings. As illustrated in Figure 6, the loss convergence remains smooth and consistent across all tested values ( $M = 8$  to  $M = 32$ ). We observed no signs of particular training instability or divergence, confirming that our multi-batch averaging strategy effectively mitigates the potential noise introduced by small-batch sampling.

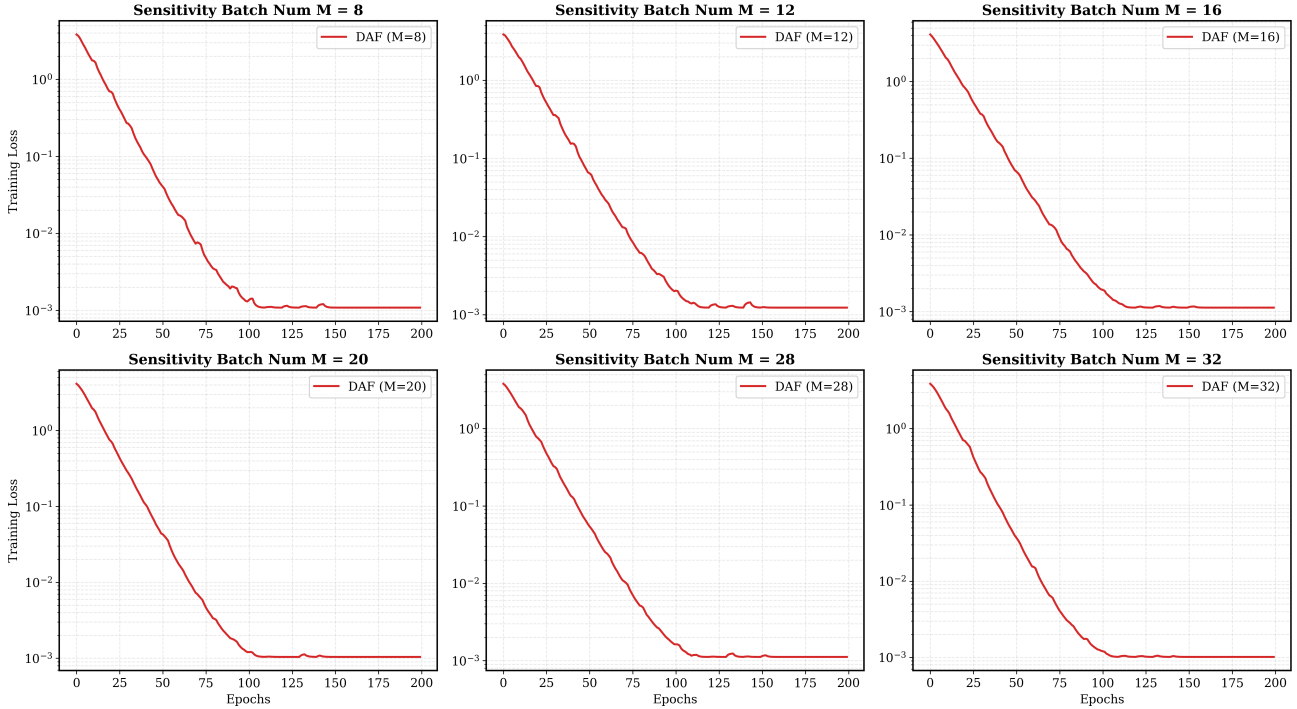


Figure 6. Training loss convergence curves on the CIFAR-100 dataset with varying Sensitivity Batch Num  $M$  ( $\{8, 12, 16, 20, 28, 32\}$ ). The overlapping curves exhibit consistent convergence behavior, demonstrating the stability of the dynamic reconfiguration process. (Curves are smoothed via exponential moving average for visual clarity.)

## I. Generalization on Different Architectures (Swin & ConvNeXt)

To demonstrate the model-agnostic advantage of DAF, we extended our evaluation to diverse architectures beyond standard ViTs, reflecting the broader importance of hierarchical, multi-level, and structured representations across learning scenarios (Zhang et al., 2025a; Liu et al., 2026; Zhang et al., 2025b). Specifically, we tested DAF on Swin-B (a hierarchical Transformer) and ConvNeXt-B (a modern CNN).

We conducted experiments on the FGVC benchmark (averaged accuracy over 5 fine-grained tasks: CUB-200, NABirds, Oxford Flowers, Stanford Dogs, and Stanford Cars). The pre-trained backbones were initialized from ImageNet-21k. For ConvNeXt, we adapted DAF by applying the dynamic selection mechanism to the pointwise convolutions ( $1 \times 1$  convs, treating them as linear layers) and normalization layers.

The comparison with Full Fine-tuning, Linear Probing, SSF (Lian et al., 2022), and GPS (Zhang et al., 2024c) is presented in Table 15.

Table 15. Performance comparisons on the FGVC benchmark (Average accuracy over 5 tasks) with different model architectures. DAF achieves the best performance with the highest parameter efficiency on both Hierarchical Transformer and CNN backbones.

| Architecture      | Swin-B       |             | ConvNeXt-B   |             |
|-------------------|--------------|-------------|--------------|-------------|
|                   | Ave. Acc.    | Params.(%)  | Ave. Acc.    | Params.(%)  |
| Full              | 92.42        | 100.00      | 93.04        | 100.00      |
| Linear            | 87.90        | 0.28        | 88.00        | 0.28        |
| SSF               | 91.54        | 0.56        | 92.48        | 0.56        |
| GPS               | 92.56        | 0.95        | 93.32        | 0.90        |
| <b>DAF (Ours)</b> | <b>92.81</b> | <b>0.32</b> | <b>93.58</b> | <b>0.35</b> |

**Analysis of the Swin-B.** DAF outperforms GPS by +0.25% while using only 1/3 of the trainable parameters (0.32% vs. 0.95%). This confirms that DAF’s dynamic reconfiguration is highly effective for hierarchical Transformer structures.

**Analysis of the ConvNeXt-B.** On the CNN-based architecture, DAF achieves a remarkable 93.58% average accuracy, surpassing both Full Fine-tuning and GPS. This indicates that our method successfully generalizes to CNNs, leveraging the dynamic sensitivity analysis to identify critical convolutional channels.

## J. Robustness Analysis of Comparison with Static DAF Method

To rigorously verify that the performance gain of DAF stems from its dynamic reconfiguration mechanism rather than suboptimal hyperparameter settings (e.g., learning rate or parameter budget) in the static baseline, we conducted a comprehensive spectrum analysis on the Static DAF.

**Experimental Setup.** We evaluated Static DAF across a broad grid of configurations, traversing different parameter budgets  $\tau \in \{0.1, 0.2, 0.3\}$  and learning rates  $lr \in \{1e-4, 3e-4, 5e-4\}$ . This covers the potential optimal range for static fine-tuning.

**Results and Analysis.** The comparative results are visualized in Figure 7. We observe the following key findings:

- **Superiority of Dynamic Paradigm:** Regardless of the combination of learning rate and parameter budget, the performance ceiling of Static DAF consistently fails to surpass that of DAF. Even with the best-performing static configuration, a distinct gap remains compared to our dynamic method.
- **Optimization Interference in Static Allocation:** Simply increasing the parameter budget (e.g., from  $\tau = 0.2$  to  $0.3$ ) in the static setting does not yield continuous improvements and, in some cases, leads to performance degradation. This corroborates our hypothesis that activating non-critical parameters throughout the entire training process introduces optimization interference (gradient noise), which hinders the model’s convergence.

These empirical results strongly demonstrate that the limitation of the baseline lies in its static assumption—the presumption that the elite parameters identified at initialization remain optimal throughout the training lifecycle. DAF breaks this assumption by dynamically shifting the budget to capture the temporal evolution of feature importance, thereby achieving superior adaptation efficiency that cannot be replicated by merely tuning static hyperparameters.

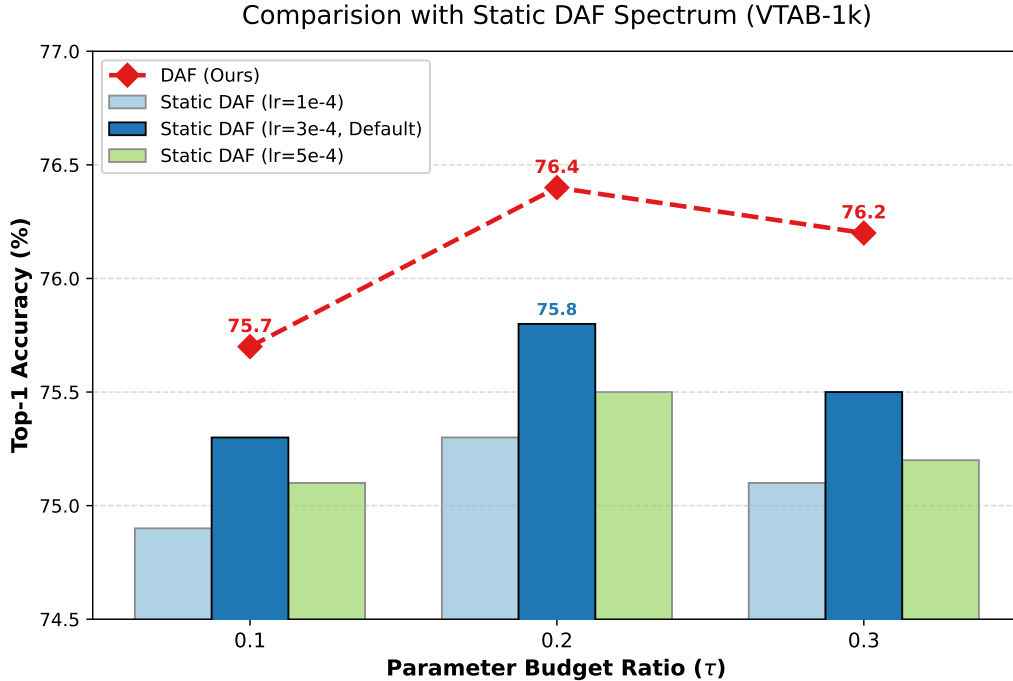


Figure 7. Performance comparison between DAF and Static DAF Spectrum on VTAB-1k. We compare DAF against Static DAF configured with various parameter budgets ( $\tau$ ) and learning rates ( $lr$ ). The red dashed line (or specific bar) represents DAF, which consistently outperforms all static configurations, validating the effectiveness of the dynamic reconfiguration paradigm.